

ELEC6910J

Deep Reinforcement Learning

Lecture 11. Temporal Difference Prediction

Course Instructor: Ling PAN

Department of Electronic and Computer Engineering

Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188, CMU 10-403, UCL COMPM050

Announcements

- HW1 and HW2 have been released (deadline: 23:59PM, Oct 16 and Oct 23)
 - Hints for both homeworks will be provided in the next week's lecture :)
- Mid-term exam: 3:00PM, Oct 23
 - Review lecture: next week

Recap: Policy Evaluation

- Iterative Policy Evaluation

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

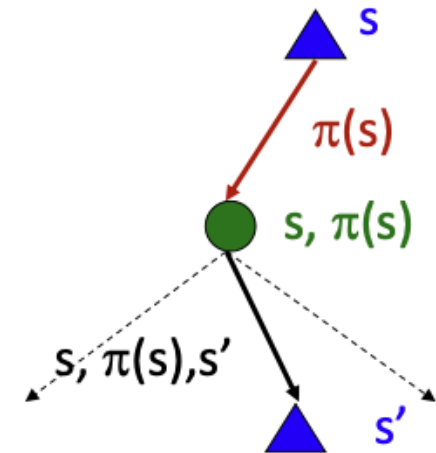
$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Do what π says to do

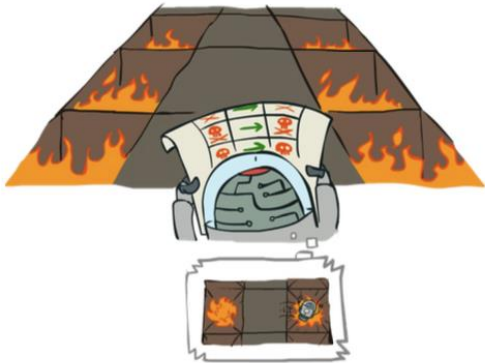


Recap: Policy Improvement

- For a given state s ,
 - would it be better to do an action $a \neq \pi(s)$ and thereafter follow π ?
- We know how good it is to follow the current policy π from $s \rightarrow V^\pi(s)$
- Would it be better or worse to change to a new policy?

Policy π : Always Go Right

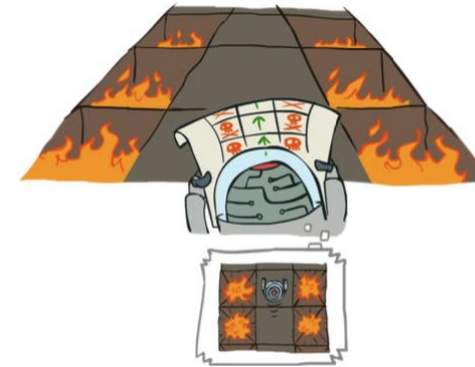
Value function V^π



-10.00	100.00	-10.00
-10.00	1.09	-10.00
-10.00	-7.88	-10.00
-10.00	-8.69	-10.00

New policy:

- Go forward
- Follow π (always go right) thereafter



Recap: Policy Iteration

- Evaluation: For fixed current policy π , find values with policy evaluation:
 - Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

- End up with value function
- Improvement: For fixed values, get a better policy using policy extraction
 - One-step look-ahead:

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

- Repeat steps until policy converges

Recap: Problems of DP

- So far, to estimate value functions we have been using dynamic programming with **known rewards and dynamics functions**:

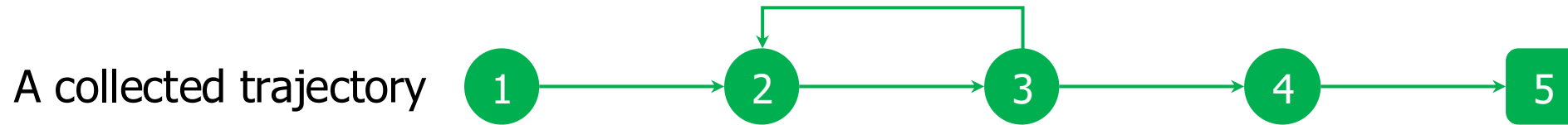
$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$$

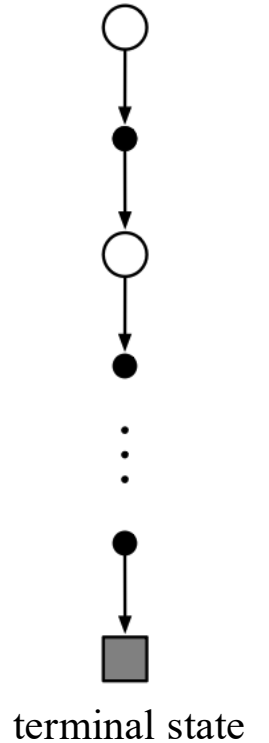
- Next: estimate value functions and policies from interaction experience, **without** known rewards or dynamics.
- How? By sampling all the way. Instead of probabilities distributions to compute expectations, we will use empirical expectations by averaging sampled returns.

Recap: Monte-Carlo Prediction

- Goal: **learn** $V^\pi(s)$ or $Q^\pi(s, a)$ from episodes of experience under policy π
- Method: average returns observed after visits to s or (s, a)



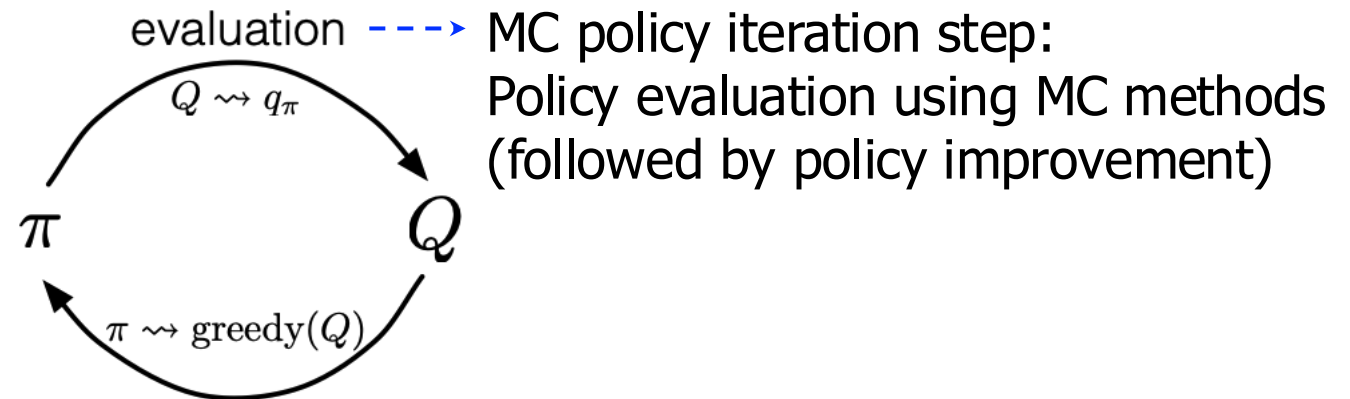
- First-visit MC: average returns only for first time s or (s, a) is visited in an episode
- Every-Visit MC: average returns for every time s or (s, a) is visited in an episode



Recap: Monte-Carlo Control

- Consider a Monte-Carlo version of classical policy iteration

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*$$



Policy improvement step: $\dashleftarrow$ improvement
Greedy with respect to Q-function

$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$

- Meaning: for each s , deterministically chooses an action with maximal Q-value
- Effect: construct each π_{k+1} as the greedy policy w.r.t. Q_k^π

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \operatorname{argmax}_a q_{\pi_k}(s, a)) \\ &= \max_a q_{\pi_k}(s, a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \end{aligned}$$

Recap: Monte-Carlo Control with Exploring Starts

- Algorithm:

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
 $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
 $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$

Recap: Monte-Carlo Control with Soft Policies

- Algorithm:

On-policy first-visit MC control (for ϵ -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\epsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ϵ -soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

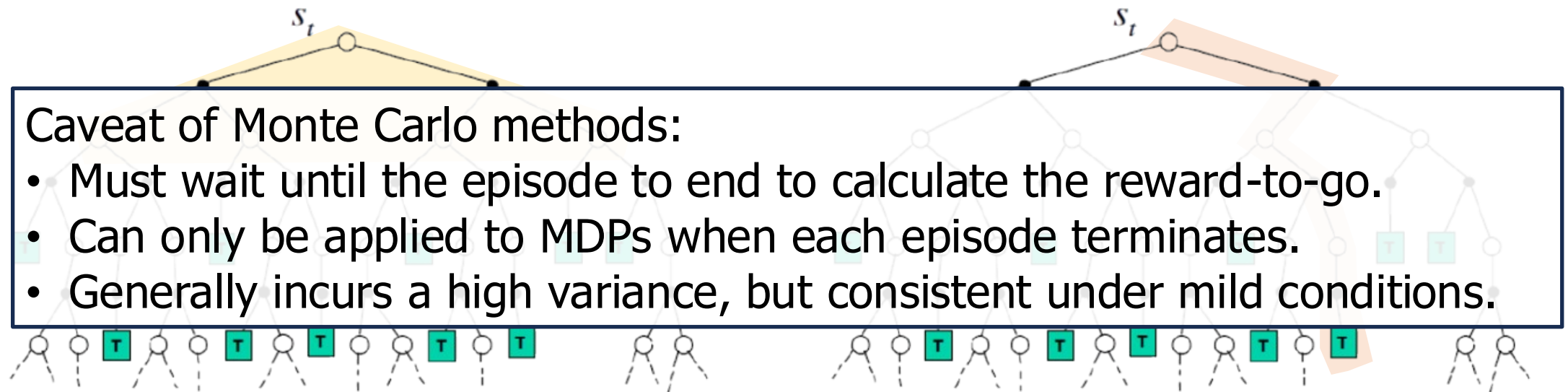
$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \epsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

→ Move it only to an ϵ -greedy policy

- Converges to the best ϵ -soft policy, but can eliminate the assumption of exploring starts

Recap: Dynamic Programming v.s. Monte Carlo

- Policy evaluation (prediction)
 - For a given policy π , compute the state-value function V^π



Dynamic Programming

$$\begin{aligned} V(S_t) &\leftarrow \mathbb{E}_\pi[R_{t+1} + \gamma V(S_{t+1})] \\ &= \sum_a \pi(a|S_t) \sum_{s', r} p(s', r|S_t, a)[r + \gamma V(s')] \end{aligned}$$

Monte-Carlo Methods

- Every-visit Monte-Carlo prediction:
$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

target: the actual return after time t

Temporal Difference (TD) Learning

“If one had to identify one idea as central and novel to RL, it would undoubtedly be TD learning.”

- TD Prediction
- TD Control



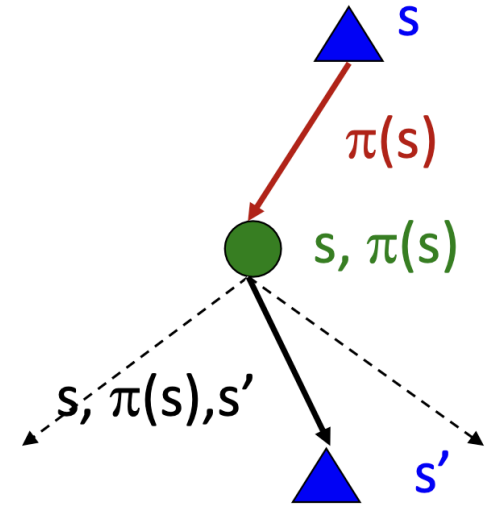
Richard Sutton

Motivation: Why Not Use Bellman Updates?

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s)]$$

- This approach fully exploited the connections between the states



Idea: Sample-based Updates

$$V_{k+1}^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^{\pi}(s)]$$

- Key question: how can we do this update to V without knowing T and R ?
 - In other words, how to we take a weighted average without knowing the weights?
- Experience it firsthand: interact with the environment and draw a sample.
 - The starting point of sample-based methods: no need to know the probability distribution, you just need to be able to sample from it.

Idea: Sample-based Updates

$$V_{k+1}^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^{\pi}(s)]$$

- Key question: how can we do this update to V without knowing T and R ?
 - In other words, how to we take a weighted average without knowing the weights?

$$V_{k+1}^{\pi}(s) \leftarrow \frac{1}{n} \sum_i \text{sample}_i$$

Idea: Sampled-Based Bellman Updates

- We want to improve our estimate of V by computing these averages:

$$V_{k+1}^{\pi}(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^{\pi}(s)]$$

- Idea: take samples of outcomes s' (by doing the action) and average

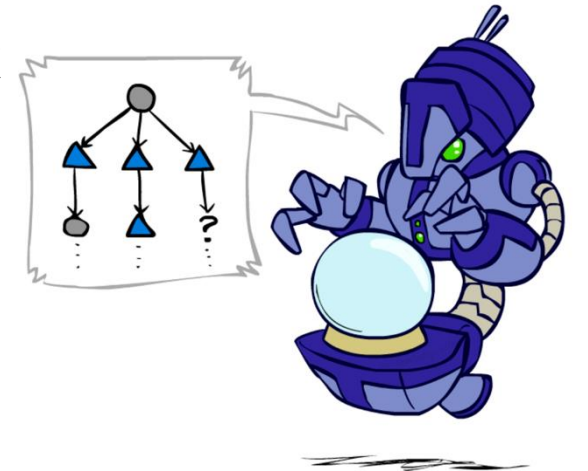
$$\text{sample}_1 = R(s, \pi(s), s'_1) + \gamma V_k^{\pi}(s'_1)$$

$$\text{sample}_2 = R(s, \pi(s), s'_2) + \gamma V_k^{\pi}(s'_2)$$

...

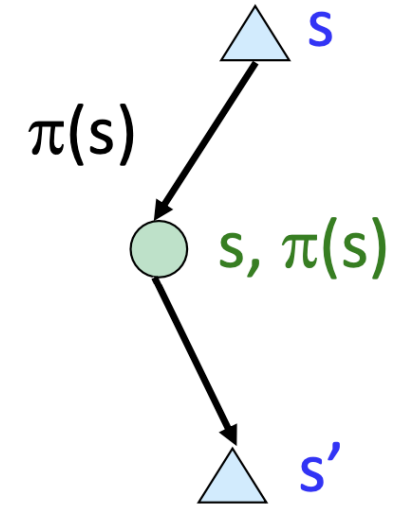
$$\text{sample}_n = R(s, \pi(s), s'_n) + \gamma V_k^{\pi}(s'_n)$$

$$\longrightarrow V_{k+1}^{\pi}(s) \leftarrow \frac{1}{n} \sum_i \text{sample}_i$$



Temporal Difference Prediction

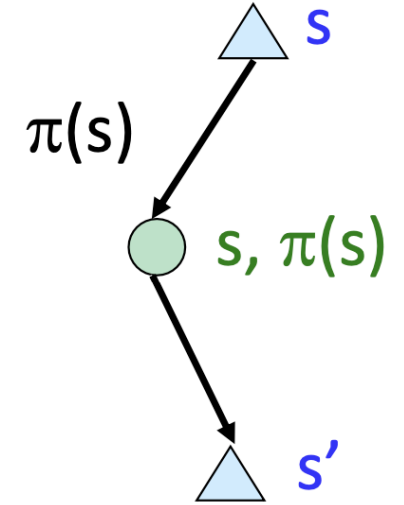
- Big idea: estimate (learn) from every experience
 - Update $V(s)$ each time we experience a transition (s, a, r, s')
 - Likely outcomes s' will contribute updates more often
- Temporal difference prediction of values
 - Policy still fixed, still doing evaluation
 - Move values toward value of whatever successor occurs: running average
 - Sample of $V(s)$: $\text{sample} = R(s, \pi(s), s') + \gamma V^\pi(s')$
 - Update to $V(s)$: $V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha \cdot \text{sample}$
 - Weighted average view: mixing old and new



Ingredient	Proportion	What it represents
 Old estimate $V^\pi(s)$	$(1 - \alpha)$	What you believed before
 New sample <i>sample</i>	α	What you just experienced
 Result		Your updated belief

Temporal Difference Prediction

- Temporal difference prediction of values
 - Policy still fixed, still doing evaluation
 - Move values toward value of whatever successor occurs: running average
 - Sample of $V(s)$: $sample = R(s, \pi(s), s') + \gamma V^\pi(s')$
 - Update to $V(s)$: $V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha \cdot sample$
 - Weighted average view: mixing old and new
 - Extreme cases
 - $\alpha = 0$: $V^\pi(s) \leftarrow V^\pi(s)$
 - The agent completely ignores new experience who never learns from new experience (stubborn).
 - $\alpha = 1$: $V^\pi(s) \leftarrow sample$
 - The agent completely forgets the old estimate and just adopts whatever it saw last (gullible).

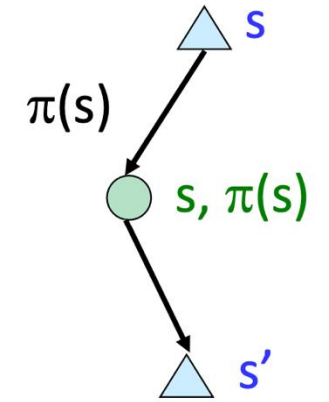


Your initial guess is: 15min

- What if you waited only 8 minutes?
- What if you waited 32 minutes?

Temporal Difference Prediction

- Temporal difference prediction of values
 - Policy still fixed, still doing evaluation
 - Move values toward value of whatever successor occurs: running average



- Sample of $V(s)$: $sample = R(s, \pi(s), s') + \gamma V^\pi(s')$
 - Update to $V(s)$: $V^\pi(s) \leftarrow (1 - \alpha)V^\pi(s) + \alpha \cdot sample$
 - Same update: $V^\pi(s) \leftarrow V^\pi(s) + \alpha (sample - V^\pi(s))$
- $\begin{matrix} \uparrow & \uparrow & \uparrow & \underbrace{\hspace{2cm}} \\ \text{New} & \text{Old} & \text{Step} & \text{Error} \\ \text{Estimate} & \text{Estimate} & \text{Size} & \end{matrix}$

- The error correction view

Temporal Difference Prediction

- Temporal difference prediction of values
 - Policy still fixed, still doing evaluation
 - Move values toward value of whatever successor occurs: running average

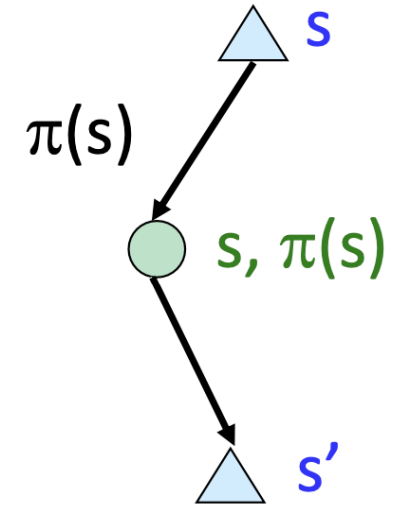
➤ Sample of $V(s)$: $\text{sample} = R(s, \pi(s), s') + \gamma V^\pi(s')$

➤ Update: $V^\pi(s) \leftarrow V^\pi(s) + \alpha(\text{sample} - V^\pi(s))$

$\begin{matrix} \text{New} & \text{Old} & \text{Step} \\ \text{Estimate} & \text{Estimate} & \text{Size} \end{matrix}$ Error

- The error correction view

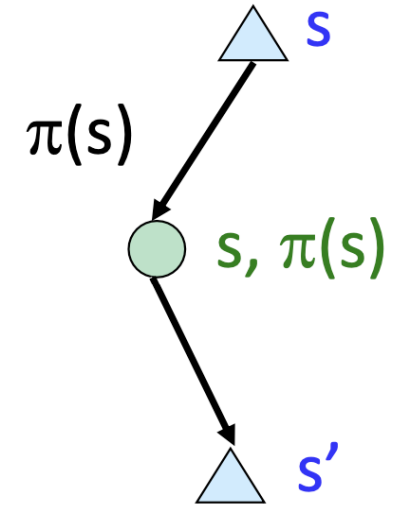
Step	What Happens	Meaning
1	Compute $\Delta = \text{sample} - V^\pi(s)$	How surprised am I?
2	Scale it: $\alpha \cdot \Delta$	How much do I want to react?
3	Add to old value	Nudge my belief accordingly



Temporal Difference Prediction

- Temporal difference prediction of values
 - Policy still fixed, still doing evaluation
 - Move values toward value of whatever successor occurs: running average
 - Sample of $V(s)$: $sample = R(s, \pi(s), s') + \gamma V^\pi(s')$
 - Update: $V^\pi(s) \leftarrow V^\pi(s) + \alpha(sample - V^\pi(s))$

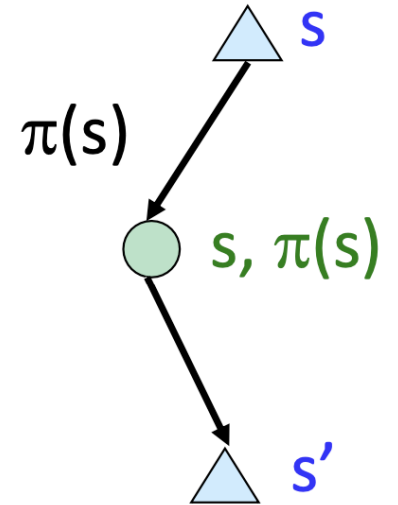
$\begin{matrix} \text{New} & \text{Old} & \text{Step} & \\ \text{Estimate} & \text{Estimate} & \text{Size} & \text{TD Error} \end{matrix}$
- The error correction view
 - $\Delta = sample - V^\pi(s)$: the temporal difference error.
 - $\Delta > 0$: Things turned out better than I expected → Nudge $V^\pi(s)$ up
 - $\Delta < 0$: Things turned out worse than I expected → Nudge $V^\pi(s)$ down
 - $\Delta = 0$: Exactly as I predicted → No update needed



Temporal Difference Prediction

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha(\text{sample} - V^\pi(s))$$

- ✓ TD methods learn directly from episodes of experience
- ✓ TD is **model-free**: no knowledge of MDP transitions / rewards
- ✓ TD learns from **incomplete** episodes, by **bootstrapping**
- ✓ TD updates a guess towards a guess

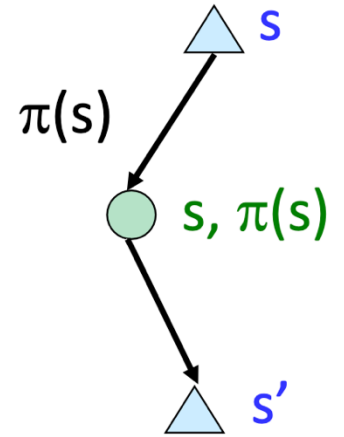


Temporal Difference Prediction

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\text{TD error } \delta_t}$$

TD target: an estimate of the return

- TD(0): one-step TD, a special case of TD(λ) and n -step TD methods
- Local updates of value on a per-action basis
- Update value $V(S_t)$ towards estimated return $R_{t+1} + \gamma V(S_{t+1})$
- Bases its update in part on an existing estimate $\rightarrow$ **bootstrapping** method (like DP)



Every-visit Monte-Carlo $V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[G_t - V(S_t)]}_{\text{TD error } \delta_t}$

- Update value $V(S_t)$ towards actual return G_t

➤ Philosophy of TD: Do not wait until the dust settles to learn, after every single step, fine-tune your belief based on what you just experienced.

Temporal Difference Prediction

Temporal Difference $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

Every-visit Monte-Carlo $V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$

- TD can learn **before** knowing the final outcome
 - TD can learn online after every step
 - MC must wait until end of episode before return is known
- TD can learn **without** the final outcome
 - TD can learn from incomplete sequences
 - MC can only learn from complete sequences
 - TD works in continuing (non-terminating) environments
 - MC only works for episodic (terminating) environments

Temporal Difference Prediction

MC:

- The expected value is unknown
- A sample return is used in place of the real expected return

$$\begin{aligned} v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \end{aligned}$$

TD: combine the sampling of MC and bootstrapping of DP

- Sample the expected value
- Use a current estimate $V(S_{t+1})$ of the true unknown $v_{\pi}(S_{t+1})$

DP:

- The expected values are provided by a model
- Use a current estimate $V(S_{t+1})$ of the true unknown $v_{\pi}(S_{t+1})$

Temporal Difference Prediction

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Initialize $V(s)$ arbitrarily (e.g., $V(s) = 0$, for all $s \in \mathcal{S}^+$)

Repeat (for each episode):

Initialize S

Repeat (for each step of episode):

$A \leftarrow$ action given by π for S

Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

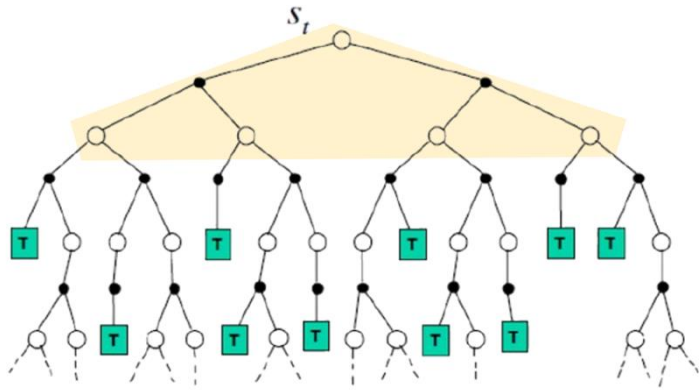
until S is terminal

Agent program

Environment program

Experiment program

Comparison

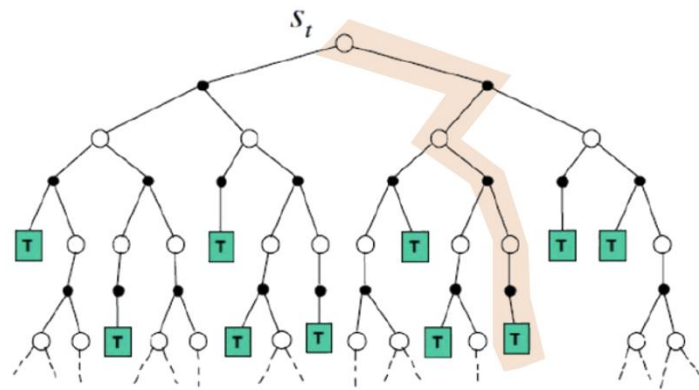


bootstrapping

Dynamic Programming

$$V(S_t) \leftarrow \mathbb{E}_{\pi}[R_{t+1} + \gamma V(S_{t+1})]$$

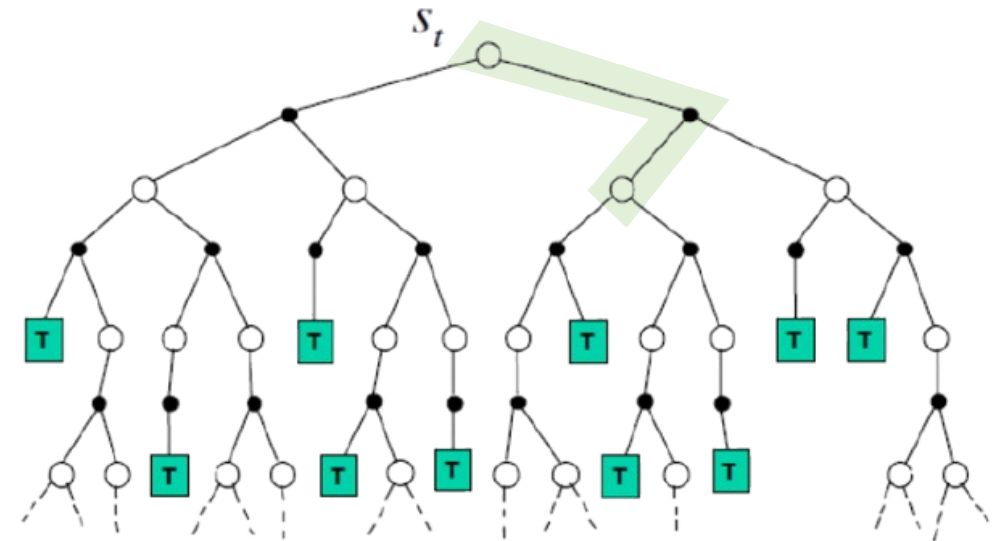
$$= \sum_a \pi(a|S_t) \sum_{s', r} p(s', r|S_t, a)[r + \gamma V(s')]$$



sampling

Monte-Carlo Methods

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)] \text{ (every visit)}$$



Temporal Difference (TD (0))

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

bootstrapping

+

sampling

Bias v.s. Variance

❖ Monte-Carlo: update value $V(S_t)$ towards actual return G_t

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

- Return $G_t = R_{t+1} + \gamma R_{t+1} + \dots + \gamma^{T-1} R_T$ is an unbiased estimate of $V^\pi(s_t)$

❖ Temporal Difference: update value $V(S_t)$ towards estimated return $R_{t+1} + \gamma V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- True TD target $R_{t+1} + \gamma V^\pi(s_{t+1})$ is an unbiased estimate of $V^\pi(s_t)$
- TD target $R_{t+1} + \gamma V(s_{t+1})$ is a biased estimate of $V^\pi(s_t)$

Bias v.s. Variance

❖ Monte-Carlo: update value $V(S_t)$ towards actual return G_t

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

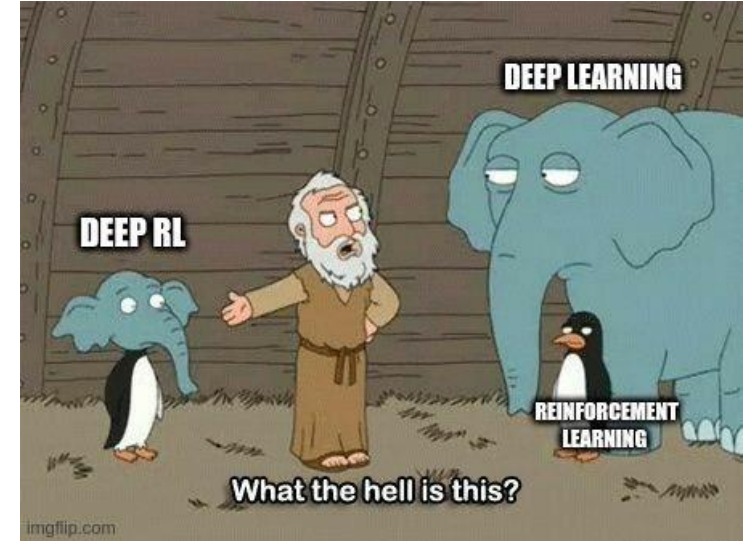
❖ Temporal Difference: update value $V(S_t)$ towards estimated return $R_{t+1} + \gamma V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- What about variance?
 - TD has lower variance than MC
 - MC return depends on many random actions, transitions, rewards
 - TD target depends on one random action, transition, reward

Bias v.s. Variance

- MC has high variance, zero bias
 - Good convergence properties
 - Not very sensitive to initial value
 - Very simple to understand and use
- TD has low variance, some bias
 - Usually more efficient than MC
 - TD(0) converges to $V^\pi(s)$ (but not always with function approximation)
 - More sensitive to initial value



Temporal Difference Prediction

Temporal Difference $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

(Every-visit) Monte-Carlo $V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$

- TD can learn **before** knowing the final outcome
 - TD can learn online after every step
 - MC must wait until end of episode before return is known
- TD can learn **without** the final outcome
 - TD can learn from incomplete sequences
 - MC can only learn from complete sequences
 - TD works in continuing (non-terminating) environments
 - MC only works for episodic (terminating) environments

Comparison: TD Methods Bootstrap and Sample

- Bootstrapping: update involves an estimate
 - MC does not bootstrap
 - DP bootstraps
 - TD bootstraps
- Sampling: update does not involve an expected value
 - MC samples
 - DP does not sample
 - TD samples

Temporal Difference Prediction

- Example: Lake (learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.5$)

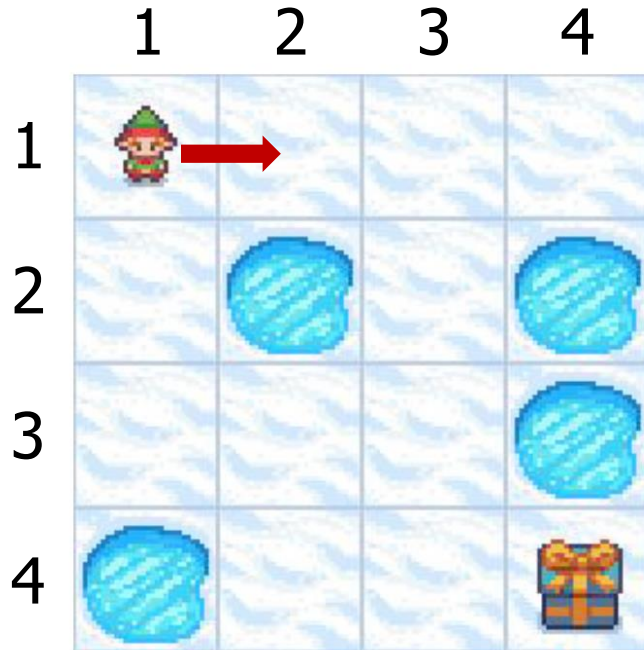


State	Value
(1, 1)	0
(1, 2)	0
(1, 3)	0
...	0
(1, 4)	0

Initialize the value function

Temporal Difference Prediction

- Example: Lake (learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.5$)



State	Value
(1, 1)	-0.04
(1, 2)	0
(1, 3)	0
...	0
(1, 4)	0

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

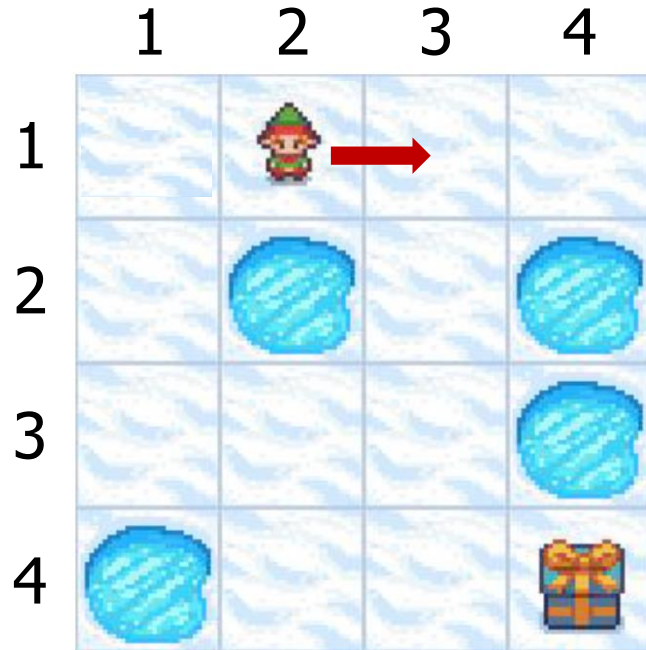
$$\begin{aligned} V(s) &= 0 + 0.1[-0.4 + 0.5 \times 0 - 0] \\ &= -0.04 \end{aligned}$$

Start from $s = (1, 1)$, and take an action right

→ Moves to the next state $s' = (1, 2)$ and receives a reward of $r = -0.4$

Temporal Difference Prediction

- Example: Lake (learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.5$)



State	Value
(1, 1)	-0.04
(1, 2)	-0.04
(1, 3)	0
...	0
(1, 4)	0

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

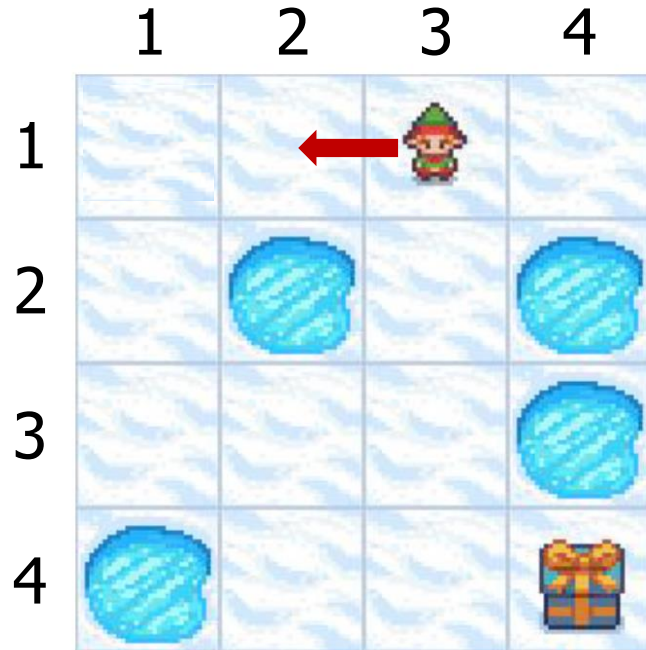
$$\begin{aligned} V(s) &= 0 + 0.1[-0.4 + 0.5 \times 0 - 0] \\ &= -0.04 \end{aligned}$$

Now we are in $s = (1, 2)$, and take an action right

→ Moves to the next state $s' = (1, 3)$ and receives a reward of $r = -0.4$

Temporal Difference Prediction

- Example: Lake (learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.5$)



State	Value
(1, 1)	-0.04
(1, 2)	-0.04
(1, 3)	-0.042
...	0
(1, 4)	0

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

$$\begin{aligned} V(s) &= 0 + 0.1[-0.4 + 0.5 \times (-0.04) - 0] \\ &= -0.042 \end{aligned}$$

Now we are in $s = (1, 3)$, and take an action left

→ Moves to the next state $s' = (1, 2)$ and receives a reward of $r = -0.4$

Temporal Difference Prediction

$$V(S_t) \leftarrow V(S_t) + \alpha \overbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}^{\text{TD error } \delta_t}$$

- Algorithm:

TD(0): one-step TD, a special case of TD(λ) and n -step TD methods

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

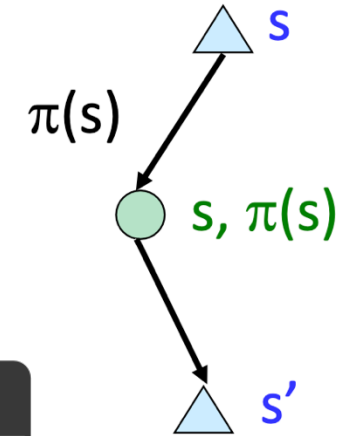
$A \leftarrow$ action given by π for S

 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$

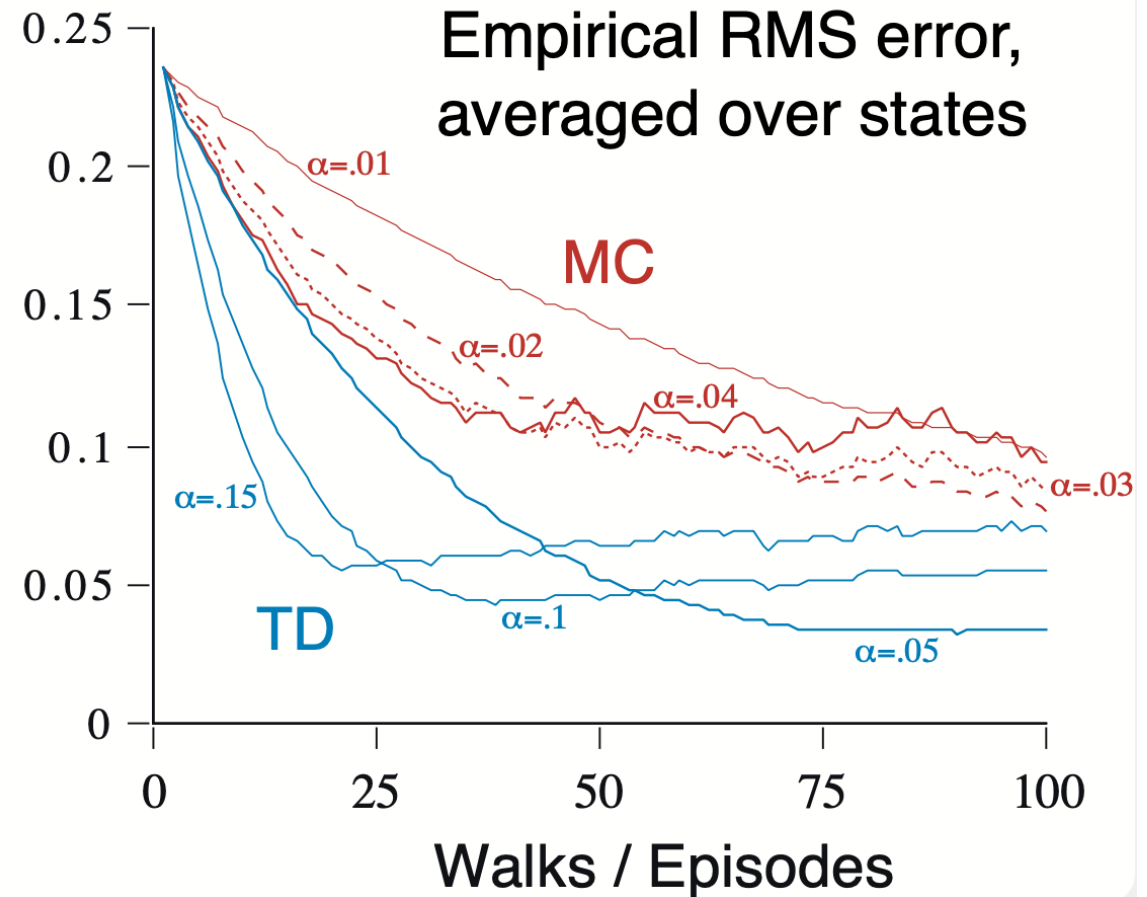
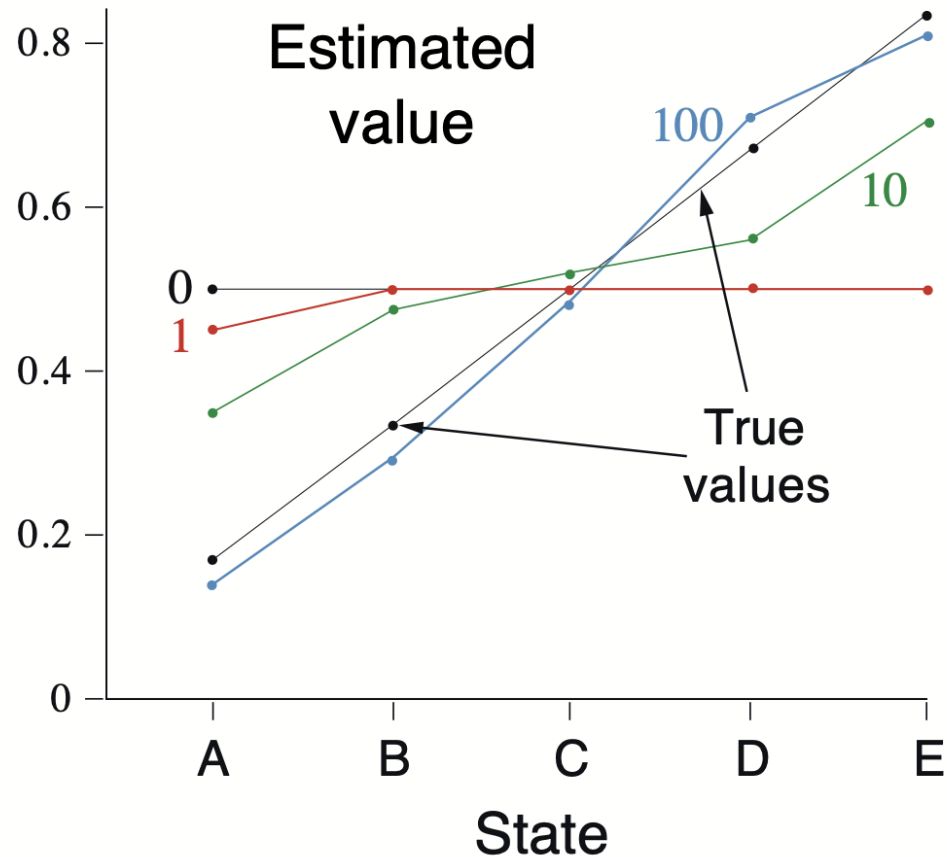
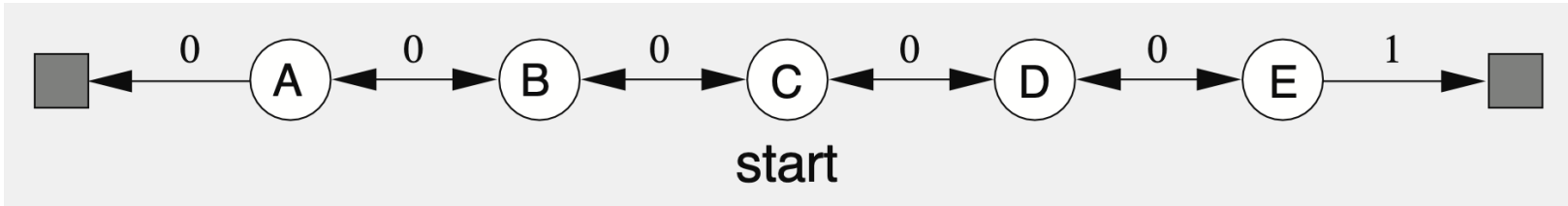
$S \leftarrow S'$

 until S is terminal

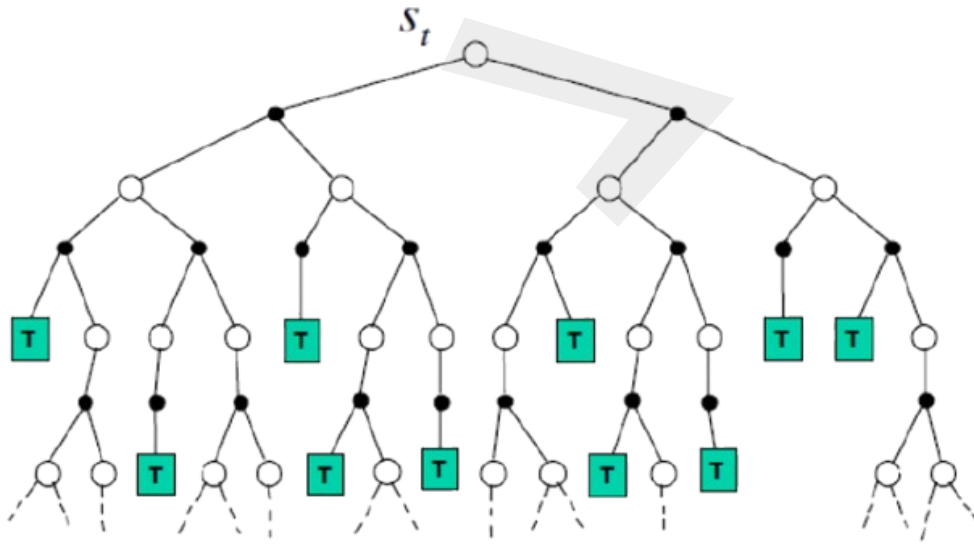


Temporal Difference Prediction

- Example:



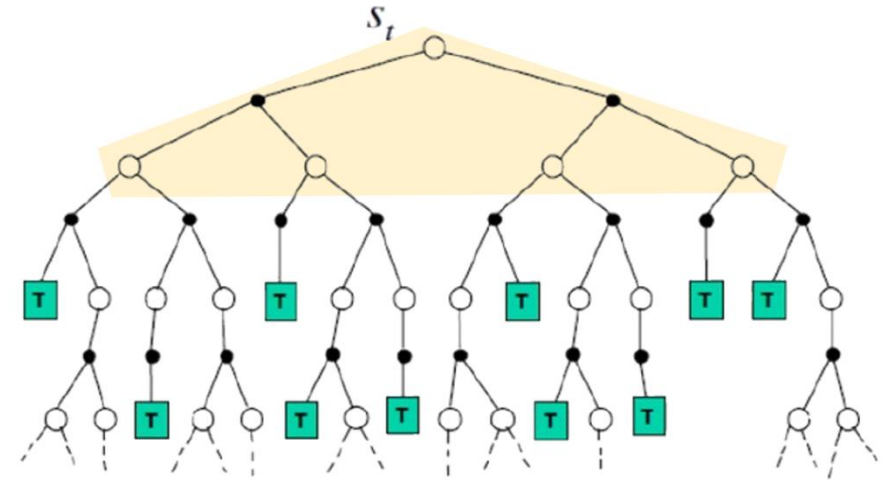
Temporal Difference Prediction



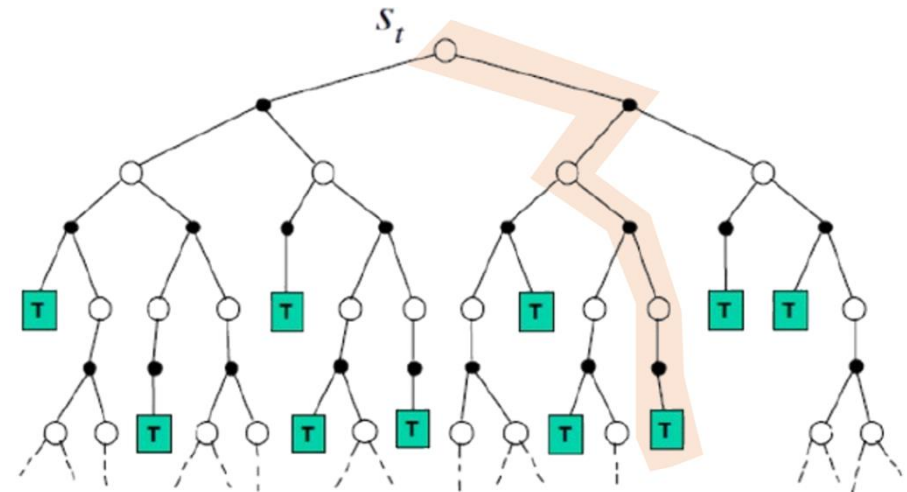
TD(0)

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Introduced one-step tabular model-free TD methods
- These methods bootstrap and sample, combining aspects of DP and MC

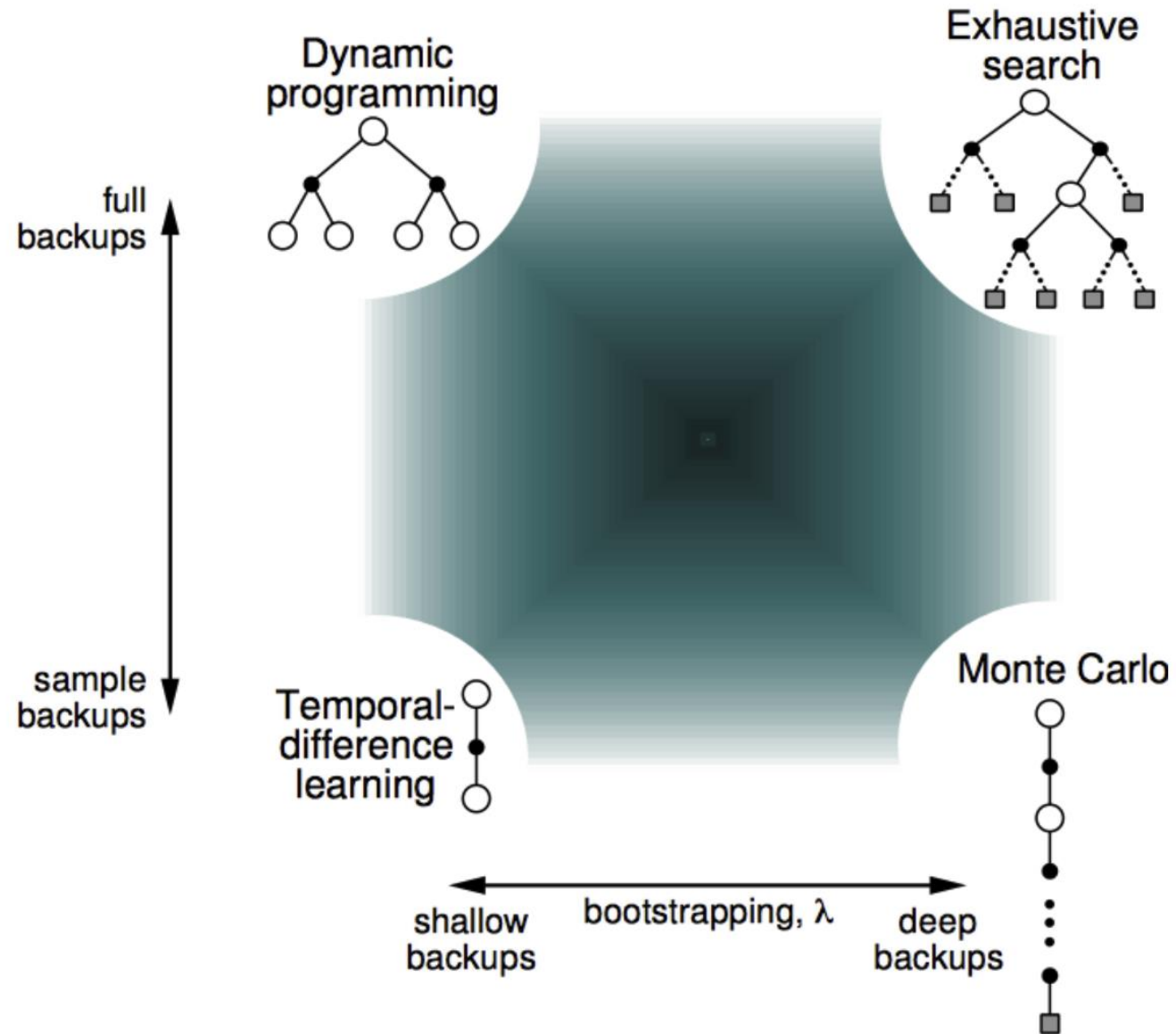


DP



MC

Unified View of Reinforcement Learning



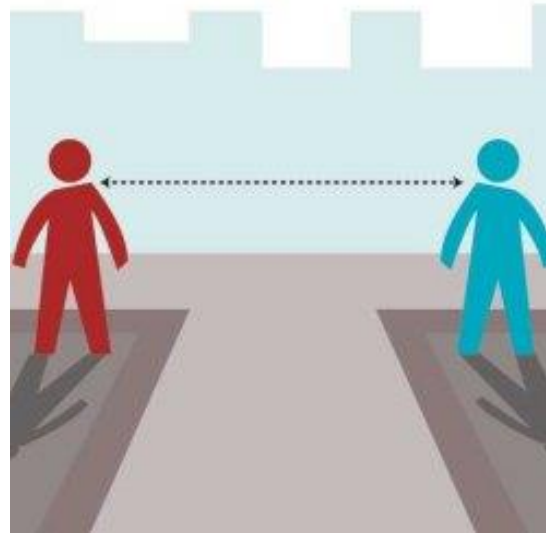
Unifying MC and TD(0)

- What is the space of methods lying between Monte Carlo and TD methods?

The update is based on just one next reward, **bootstrapping** from the value of the state **one step later** as a proxy for the remaining rewards.

$$R_{t+1} + \gamma V(S_{t+1})$$

1-step TD
and TD(0)



Any intermediates?

∞ -step TD
and Monte Carlo



Perform an update for each state based on the **entire** sequence of observed rewards from that state **until the end of the episode**.

G_t

Unifying MC and TD(0)

- What is the space of methods lying between Monte Carlo and TD methods?

Perform an update based on an intermediate number of rewards: more than one, but less than all of them until termination.

Unifying MC and TD(0)

- What is the space of methods lying between Monte Carlo and TD methods?

The update is based on just one next reward, bootstrapping from the value of the state one step later as a proxy for the remaining rewards.

$$R_{t+1} + \gamma V(S_{t+1})$$

1-step TD
and TD(0)



2-step TD



Based on the first two rewards and the estimated value of the state two steps later

∞ -step TD
and Monte Carlo



Perform an update for each state based on the entire sequence of observed rewards from that state until the end of the episode.

G_t

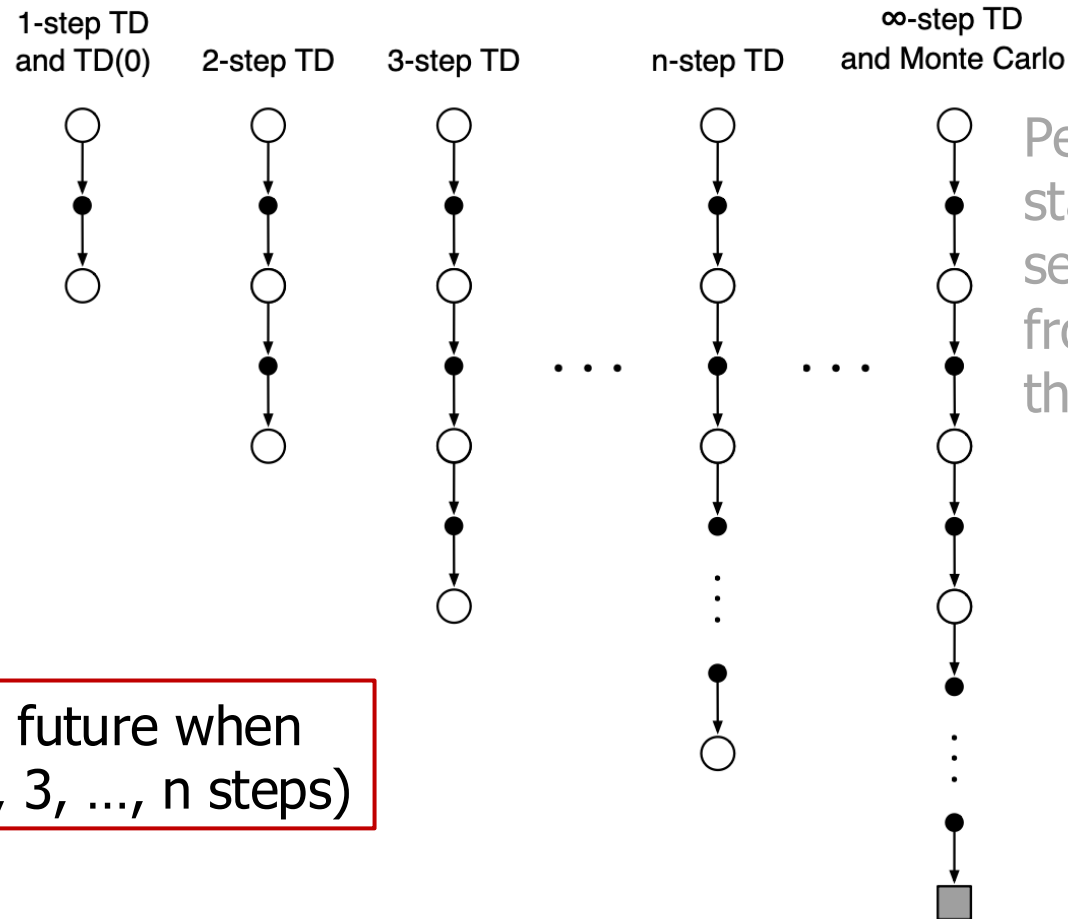
Unifying MC and TD(0)

- What is the space of methods lying between Monte Carlo and TD methods?

The update is based on just one next reward, bootstrapping from the value of the state one step later as a proxy for the remaining rewards.

$$R_{t+1} + \gamma V(S_{t+1})$$

Idea: look farther into the future when you do TD – backup (1, 2, 3, ..., n steps)



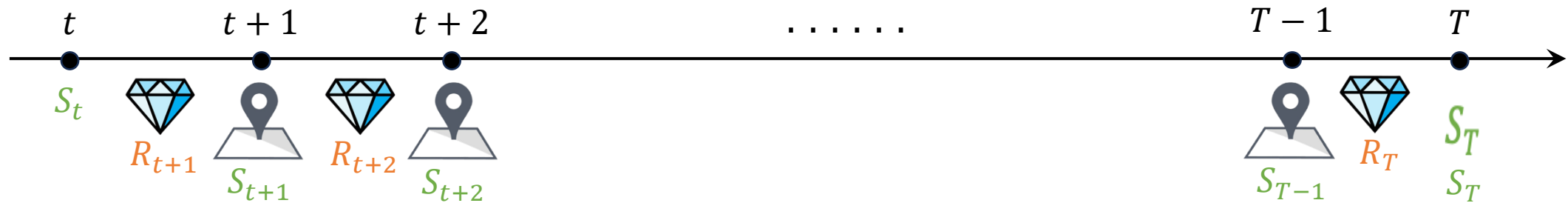
→ These methods form a spectrum ranging from 1-step TD methods to Monte Carlo methods. 44

n -Step TD Prediction

- Monte Carlo: updates the estimate of $v_\pi(S_{t+1})$ in the direction of the complete return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$$

- T : the last step of the episode



n -Step TD Prediction

- One-step TD:

Use $V_t(S_{t+1})$ to estimate
the remaining return

$$G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})$$

One-step reward
(reward until time $t + 1$)

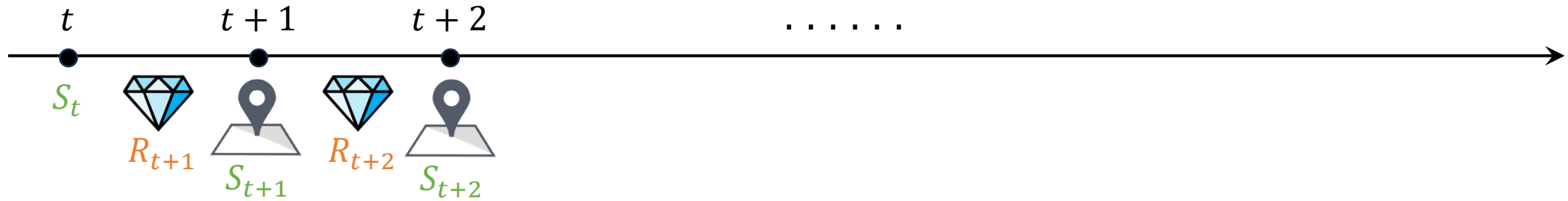
- $G_{t:t+1}$: a truncated return for time t using rewards up until time $t + 1$
 - Use $V_t(S_{t+1})$ to estimate the remaining return



n -Step TD Prediction

- Two-step TD:

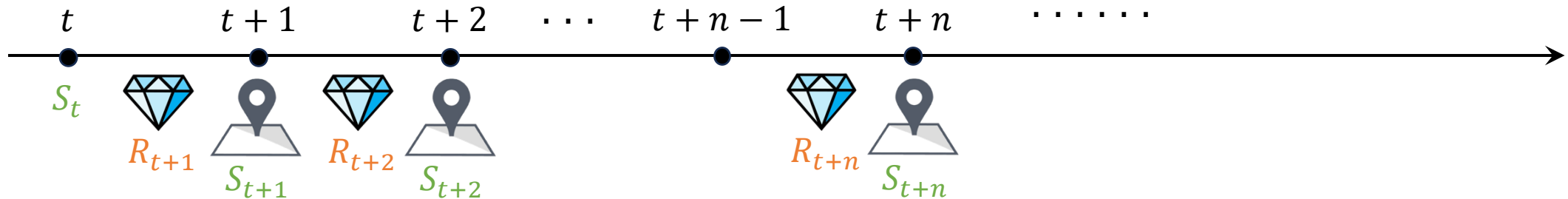
$$G_{t:t+2} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$$



- n -step TD:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- $\forall n, t$ such that $n \geq 1$ and $0 \leq t < T - n$

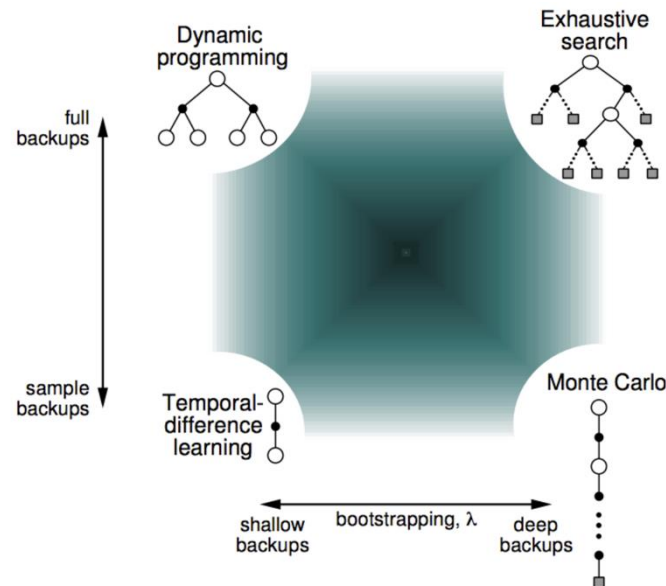


n -Step TD Prediction

- n -step TD:

$$V(S_t) \leftarrow V(S_t) + \alpha(G_{t:t+n} - V(S_t))$$

- Error reduction property
- All n -step TD methods converge to the correct predictions under appropriate technical conditions.
- The n -step TD methods thus form a family of sound methods, with one-step TD methods and Monte Carlo methods as extreme members.



Model-free Estimation

Problem: we don't know P or R



Which states/actions are good?

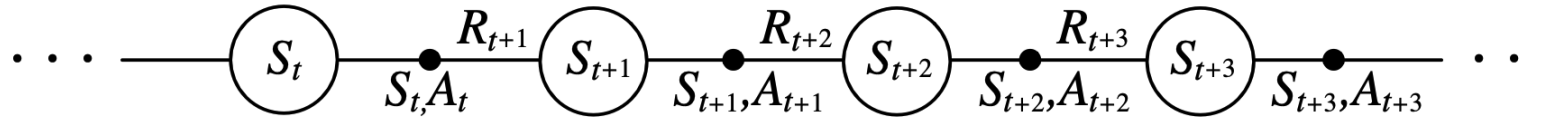
→ Learn by **trial and error**

➤ Value function: $V^\pi(s)$ / $Q^\pi(s, a)$



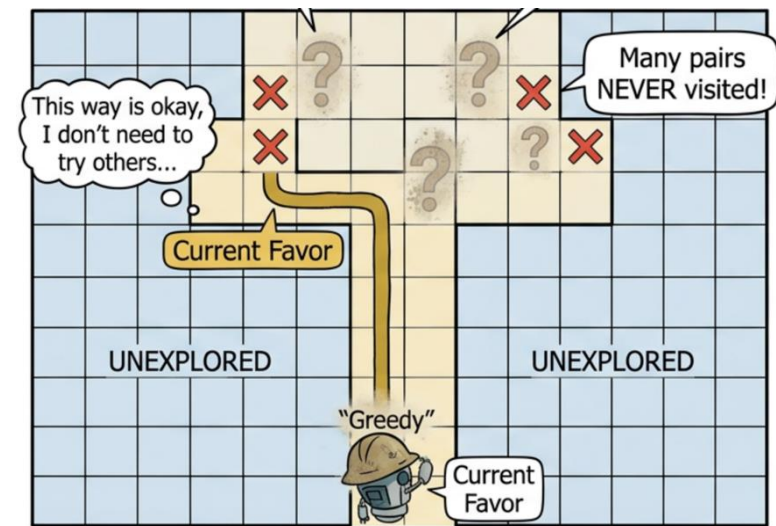
Learning Action-Value Function

- Estimate $Q^\pi(s, a)$ for the current policy



An episode consists of an alternating sequences of state and state-action pairs

- Update rule: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$
 - The update is done after every transition from a non-terminal state S_t
 - If S_{t+1} is terminal, then define $Q(S_{t+1}, A_{t+1}) = 0$
- Requires a behavior policy to collect data for learning
 - Needs to balance exploration and exploitation
e.g., epsilon-greedy



ELEC6910J

Deep Reinforcement Learning

Lecture 12. Temporal Difference Control

Course Instructor: Ling PAN

Department of Electronic and Computer Engineering

Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188/CS287, CMU 10-403, UCL COMPM050

Temporal Difference (TD) Learning

“If one had to identify one idea as central and novel to RL, it would undoubtedly be TD learning.”

- TD Prediction
- **TD Control**



Richard Sutton

SARSA: On-Policy TD Control

- Based on TD prediction, how can we find the optimal policy?
 - A TD method: makes use of the TD update rule to estimate the value function

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- How does it improve the policy to obtain a greater value function?
→ The SARSA algorithm for solving the control problem (which means it tries to find an optimal policy for a given task)
- Idea:
 - Continually estimate Q^π for the behavior policy π (e.g., epsilon-greedy policy)
 - At the same time change π toward greediness w.r.t. Q^π

SARSA: On-Policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [\underset{\text{immediate}}{\underset{\text{reward}}{R_{t+1}}} + \gamma \underset{\text{future}}{\underset{\text{value}}{Q(S_{t+1}, A_{t+1})}} - Q(S_t, A_t)]$$

future • next state
value • next action

- Components:
 - State (S)
 - Action (A)
 - Reward (R)
 - Next state (S')
 - Next action (A')

SARSA: On-Policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

immediate
reward

future • next state
value • next action

- Select action using an exploration strategy (e.g., epsilon-greedy policy)
- Take the action → Observe the reward & Move to the next state
- Select the next action based on the current policy
- Update the Q-values Based on the SARSA Update Rule
- After training for a while...
- Update for the Q-value estimate for the current state-action pair (S_t, A_t) : $Q(S_t, A_t)$
 - Immediate reward (R): R_{t+1}
 - Next state (S'): S_{t+1}
 - Next action (A'): A_{t+1}

SARSA: On-Policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

immediate
reward

future • next state
value • next action

- Error correction view

Step	What Happens	Meaning
1	Compute $\Delta = \text{sample} - Q(s, a)$	How surprised am I?
2	Scale it: $\alpha \cdot \Delta$	How much do I want to react?
3	Add to old value	Nudge my belief accordingly

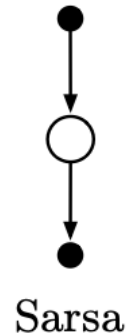
SARSA: On-Policy TD Control

- Why is called SARSA?

S, A, R, S', A' the name of SARSA comes from the transition from state S to S'

→ Reflects the fact that the main function for updating the Q-value depends on

- The current state of the agent "S"
- The action the agent chooses "A"
- The reward "R" the agent gets for choosing this action
- The state "S'" that the agent enters after taking that action
- The next action "A'" the agent chooses in its new state



SARSA: On-Policy TD Control

- Algorithm:

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

- Why is it on-policy? (will be introduced later)

- In on-policy learning, the optimal value function is learned from actions taken using the current policy $\pi(a|s)$.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

SARSA: On-Policy TD Control

- Convergence guarantee: SARSA converges conditions on *the step sizes*, as long as *all state-action pairs are visited an infinite number of times* and *the policy converges in the limit to the greedy policy*.
 - Step size conditions

Step sizes must **not decay too slowly**

Step sizes must **not decay too fast**

SARSA: On-Policy TD Control

- Convergence guarantee: SARSA converges conditions on *the step sizes*, as long as *all state-action pairs are visited an infinite number of times* and *the policy converges in the limit to the greedy policy*.

- Step size conditions

Step sizes must **not decay too slowly** $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$

- If the step size α shrinks too slowly, it means the the **cumulative noise never fully vanishes**.
→ The Q-values keep jittering around the true answer, getting close but never actually settling down.

SARSA: On-Policy TD Control

- Convergence guarantee: SARSA converges conditions on *the step sizes*, as long as *all state-action pairs are visited an infinite number of times* and *the policy converges in the limit to the greedy policy*.

- Step size conditions

Step sizes must **not decay too fast** $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$

- If the step size α shrinks too fast, it means *the agent essentially stops listening before it has gathered enough experience*.

→ Obvious errors remain frozen in the Q-table.

(e.g., falling asleep with "1+1=3" still in your notes and never fixing it).

SARSA: On-Policy TD Control

- Convergence guarantee: SARSA converges conditions on *the step sizes*, as long as *all state-action pairs are visited an infinite number of times* and *the policy converges in the limit to the greedy policy*.

- Step size conditions

- All state-action pairs are explored infinitely many times:

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

- If some state-action pair (s, a) is only visited a handful of times or never, then its Q-value is basically a guess.
- Example: Having a chapter on the exam you never once opened. No amount of clever updating can give you a good estimate of something you have no data for.

You can't learn what you've never tried.

SARSA: On-Policy TD Control

- Convergence guarantee: SARSA converges conditions on *the step sizes*, as long as *all state-action pairs are visited an infinite number of times* and *the policy converges in the limit to the greedy policy*.

- **Step size conditions**

- **All state-action pairs are explored infinitely many times:**

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

Analogy: Every chapter must be reviewed, and not just once
→ many, many times, so the law of large numbers can kick in.

- When the agent explores thoroughly that tries every action in every state infinitely often, the Q-value estimates are backed by abundant data.
- Each $Q(s,a)$ has seen enough samples of "what happens next" to converge to the true expected return.

Every state-action pair, visited again and again.

SARSA: On-Policy TD Control

- Convergence guarantee: SARSA converges conditions on *the step sizes*, as long as *all state-action pairs are visited an infinite number of times* and *the policy converges in the limit to the greedy policy*.
 - Step size conditions
 - All state-action pairs are explored infinitely many times
 - The policy converges in the limit to the greedy policy:
$$\lim_{k \rightarrow \infty} \pi_k(a|s) = \mathbb{I}(a = \operatorname{argmax}_{a' \in A} Q_k(s, a'))$$
 - For example, ϵ -greedy is GLIE (greedy in the limit) policies by setting $\epsilon = \frac{1}{t}$ (ϵ reduces to 0)

SARSA: On-Policy TD Control

- Convergence guarantee: SARSA converges conditions on the step sizes, as long as *all state-action pairs are visited an infinite number of times and the policy converges in the limit to the greedy policy*

- Step size conditions

- Step sizes must not decay too fast
 - Step sizes must not decay too slowly

Just for better understanding



- All state-action pairs are explored infinitely many times:

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

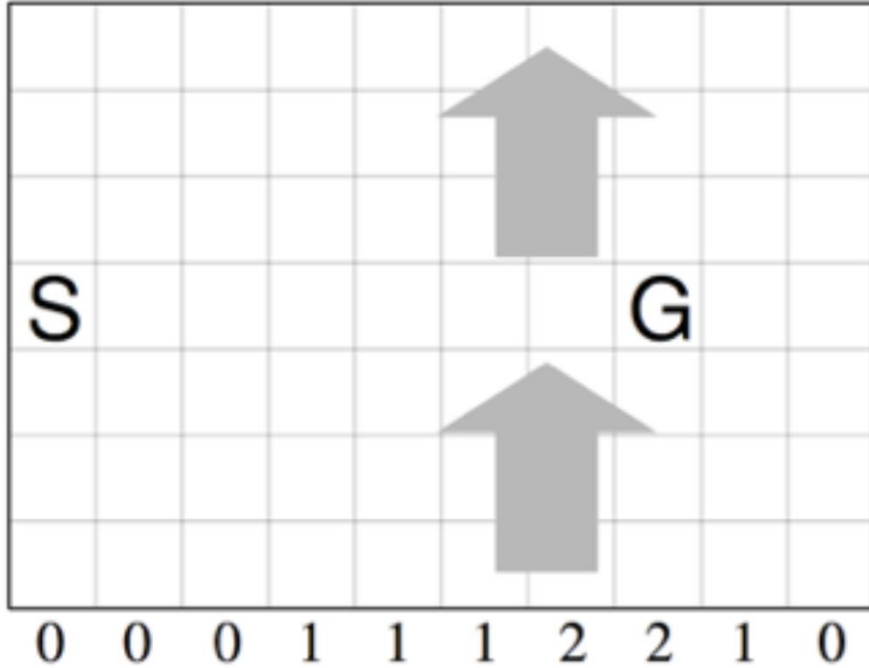
- The policy converges in the limit to the greedy policy:

$$\lim_{k \rightarrow \infty} \pi_k(a|s) = \mathbb{I}(a = \operatorname{argmax}_{a' \in A} Q_k(s, a'))$$

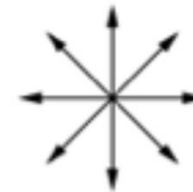
- For example, ϵ -greedy is GLIE (greedy in the limit) policies by setting $\epsilon = \frac{1}{t}$ (ϵ reduces to 0)

SARSA: On-Policy TD Control

- Example: Windy GridWorld



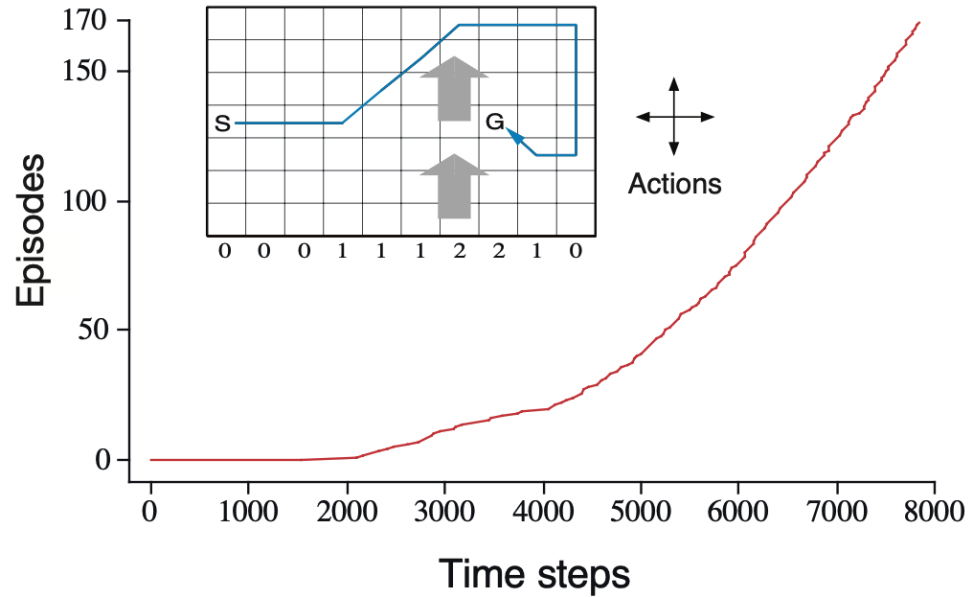
standard
moves



king's
moves

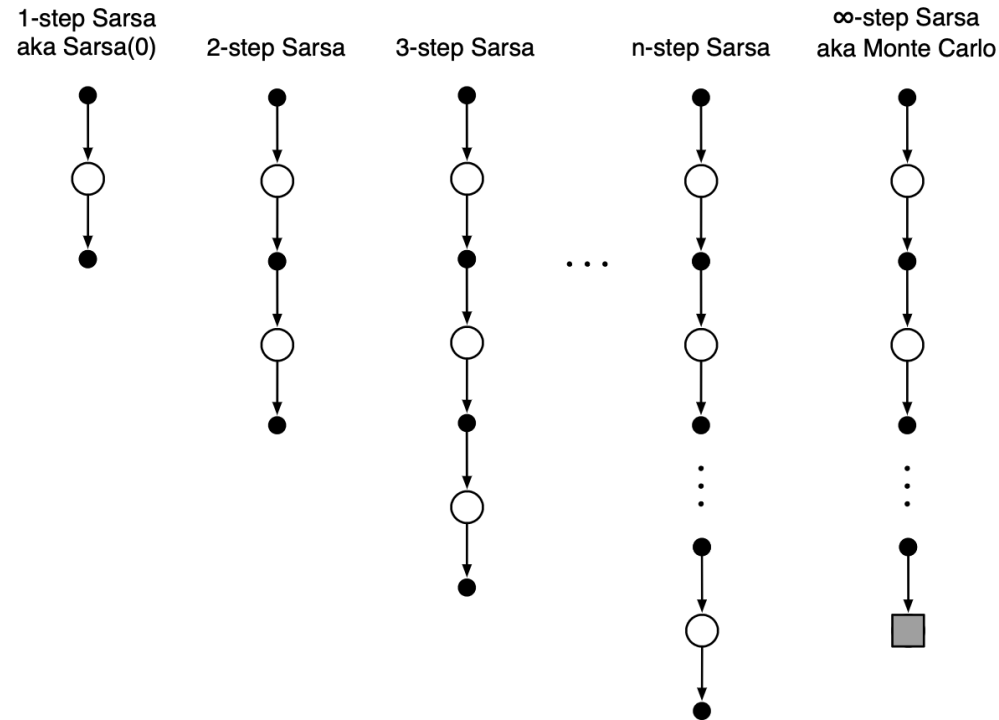
SARSA: On-Policy TD Control

- Example: Windy GridWorld



n -step SARSA

- Previous version: one-step Sarsa
- How can n -step methods be used not just for prediction, but for control?
 - Model-free control $\rightarrow$ use $Q(s, a)$, not $V(s)$ (different from n -step TD prediction)
- Idea: switch states for actions (state-action pairs) and then use an ϵ -greedy policy

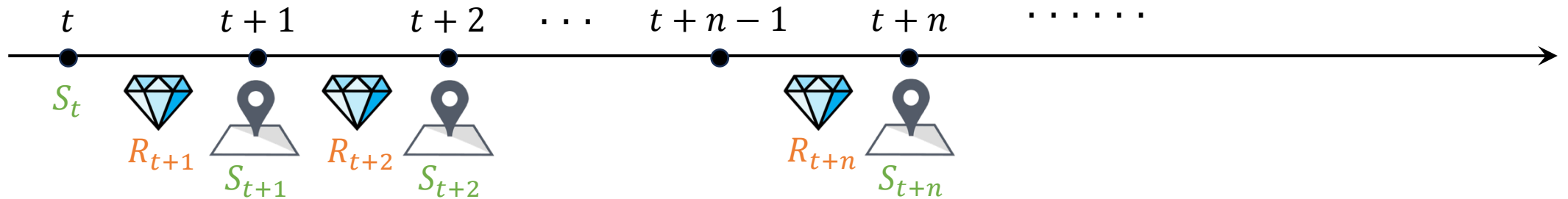


n -step SARSA

- n -step update targets in terms of estimated action values:

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q_{t+n-1}(S_{t+n}, A_{t+n}), \quad n \geq 1, 0 \leq t < T-n$$

- $G_{t:t+n} = G_t$ if $t+n > T$



- Algorithm:

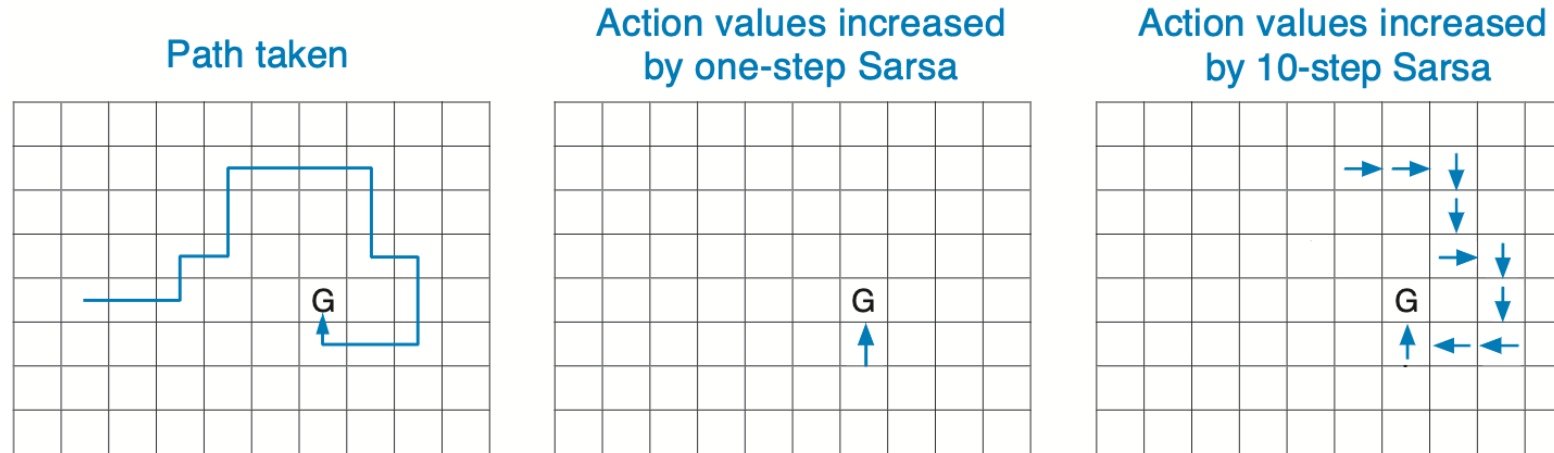
$$Q_{t+n}(S_t, A_t) \doteq Q_{t+n-1}(S_t, A_t) + \alpha [G_{t:t+n} - Q_{t+n-1}(S_t, A_t)], \quad 0 \leq t < T,$$

- The values of all other states remain unchanged:

$$Q_{t+n}(s, a) = Q_{t+n-1}(s, a), \quad \forall s, a \text{ such that } s \neq S_t \text{ and } a \neq A_t$$

n -step SARSA

- What's the advantage of $n > 1$? → may speedup policy learning
- Reward function: all rewards were zero except for a positive reward at G
- Initialization: the values were all initially 0



The path taken by an agent in a single episode (ending at a location of high reward)

- Arrows: which action values were strengthened as a result of this path
- One-step Sarsa: strengthens only the last action of the sequence of actions that led to the high reward
- n -step Sarsa: strengthens the last n actions of the sequence, so that much more is learned from the one episode

Bellman Optimality Equation

- Bellman optimality equation (recursive definition) for Q:

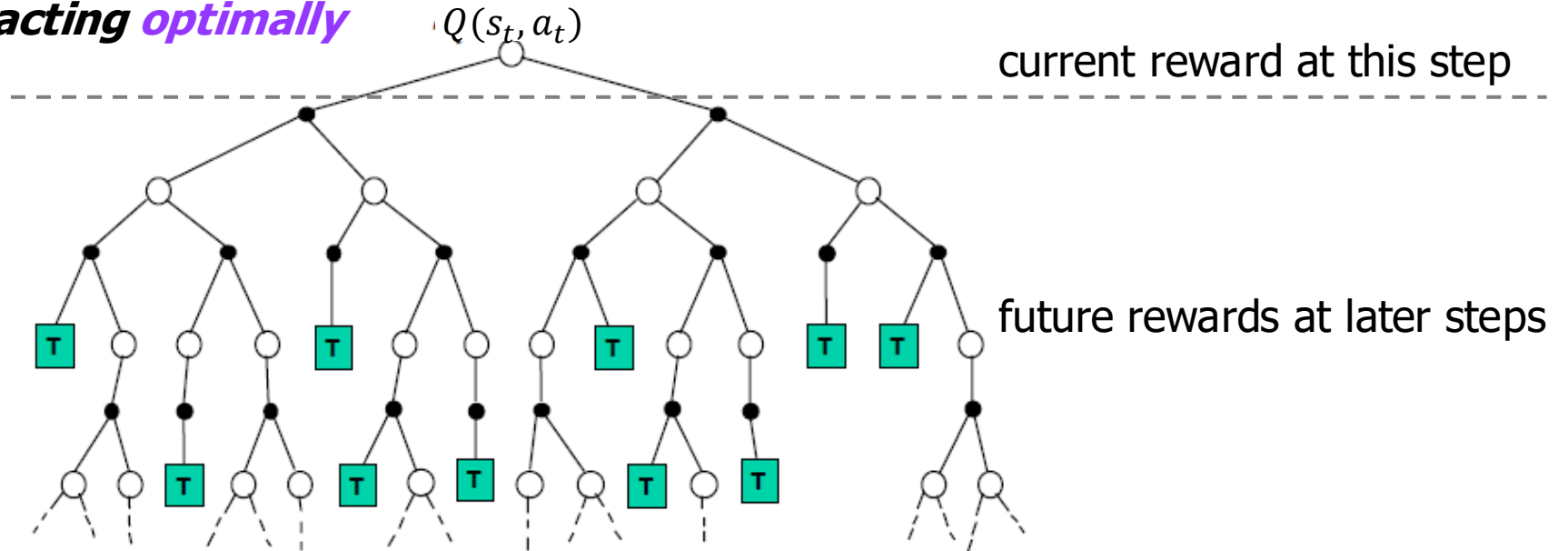
State-action value

$$Q(s, a) \leftarrow \sum_{s'} P(s, a, s') [\underbrace{R(s, a, s')}_{\text{Current}} + \gamma \underbrace{\max_{a'} Q(s', a')}_{\text{Future}}]$$

Transition probability
Immediate reward
Future (discounted) rewards

all possible next states!

❖ Def: **expected** return starting in s , taking action a , and (**thereafter**) **acting optimally**



Q-value Iteration

- Bellman optimality equation:

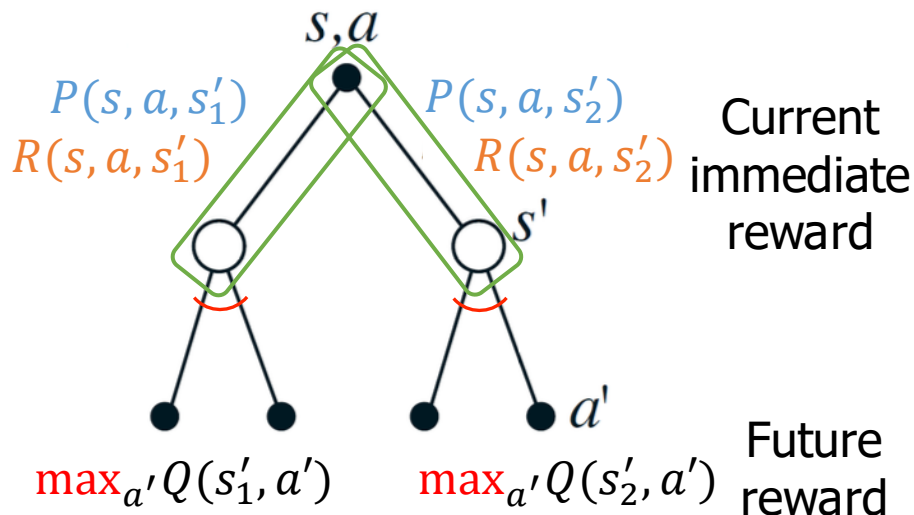
$$Q^*(s, a) = \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

- Q-value iteration: $Q_{k+1}(s, a) \leftarrow \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q_k(s', a') \right]$



Rewrite as expectation

$$Q_{k+1} \leftarrow \mathbb{E}_{s' \sim P(s'|s, a)} [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')]$$



- Problem: cannot compute the update without the knowledge of P and R
- Idea: use actual **samples** to **approximate the expectation**

↳ $\sim$ drawn from P

Q-learning: Off-Policy TD Control

- Algorithm:

Q-learning: **sampled-based** Q-value iteration

- Learn $Q(s, a)$ as you go

- Receive a sample (s, a, r, s')

- Compute the current sample: $\text{sample} = \underbrace{R(s, a, s')}_{\text{immediate reward}} + \gamma \underbrace{\max_{a'} Q(s', a')}_{\text{future returns}}$

- Update to $Q(s, a)$ by **running average**: $\underbrace{Q(s, a)}_{\text{new estimate}} \leftarrow (1 - \underbrace{\alpha}_{\text{learning rate}}) \underbrace{Q(s, a)}_{\text{old estimate}} + \underbrace{\alpha \text{ sample}}_{\text{sampled estimate}}$
weighted combination

- ✓ Effect: Move values toward value of whatever successor occurs
- ✓ Theoretical guarantee: Convergence

Q-learning: Off-Policy TD Control

- 1 Start with an initial Q-function ($|S| \times |A|$)
- 2 For episode $k = 1, 2, \dots$ till convergence
- 3 Initialize the environment to get the initial state s
- 4 For step $t = 1, 2, \dots$ till termination
- 5 Sample action a from a policy derived from Q
- 6 Take action a , observe next state s' and get reward r
- 7 Perform the update by
$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a))$$
- 8 Transition to next state $s \leftarrow s'$

Q-learning: Off-Policy TD Control

- 1 Start with an initial Q-function ($|S| \times |A|$)
- 2 For episode $k = 1, 2, \dots$ till convergence
- 3 Initialize the environment to get the initial state s
- 4 For step $t = 1, 2, \dots$ till termination
- 5 **Sample action a from a policy derived from Q**
- 6 Take action a , observe next state s' and get reward r
- 7 Perform the update by
$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a))$$
- 8 Transition to next state $s \leftarrow s'$

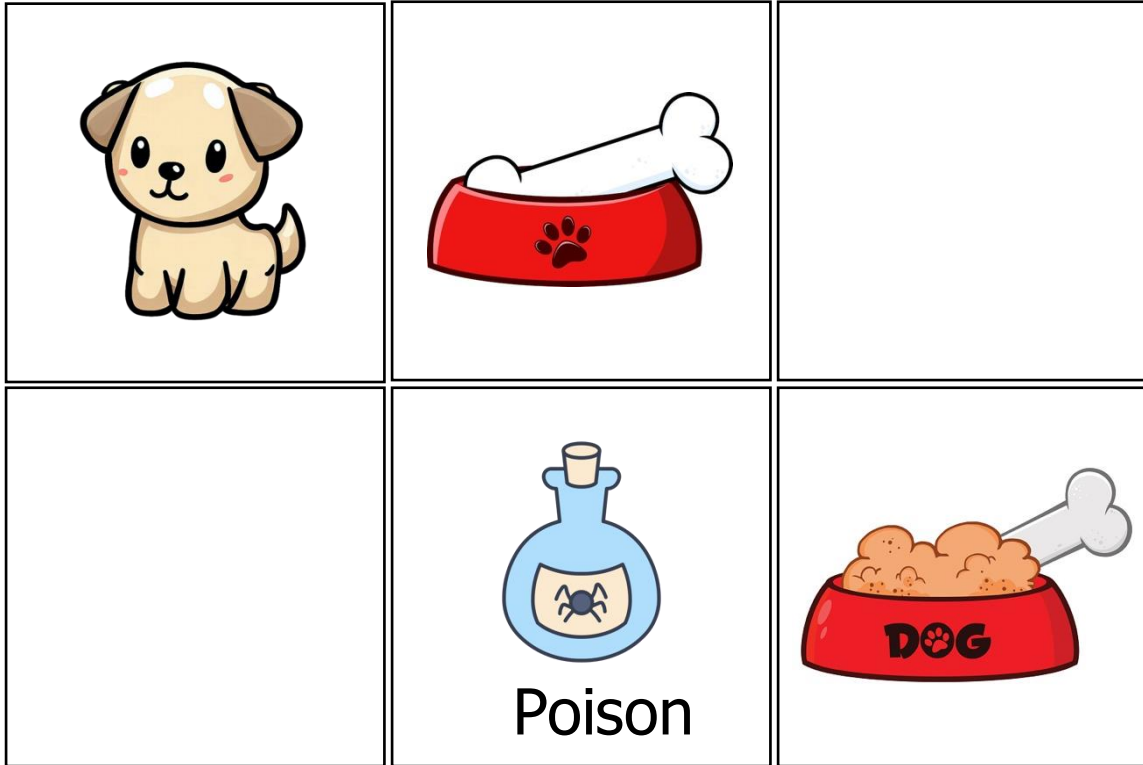
- How to sample actions?
 - Choose action that maximizes $Q_k(s, a)$ (i.e., greedily)?

Q-learning: Off-Policy TD Control

- Exploration v.s. Exploitation
 - Exploitation: To try to get reward. We exploit our current knowledge to get a payoff.
 - Exploration: Get more information about the world. How do we know if there is not a pot of gold around the corner?
- The tradeoff is important in RL and life.
 - A naïve approach: if you know enough about the world, you exploit. You know nothing, you explore.

Q-learning: Off-Policy TD Control

- Example:



Reward {
+1: eat the small bone
+10: eat the animal food with the bone
-10: eat the poison
0: otherwise



Goal: eat the animal food with the bone while avoiding the poison

- Termination: if the dog eats the poison/animal food with the bone, or it spends more than 5 steps

Q-learning: Off-Policy TD Control

- Example:

Initialize the Q-table

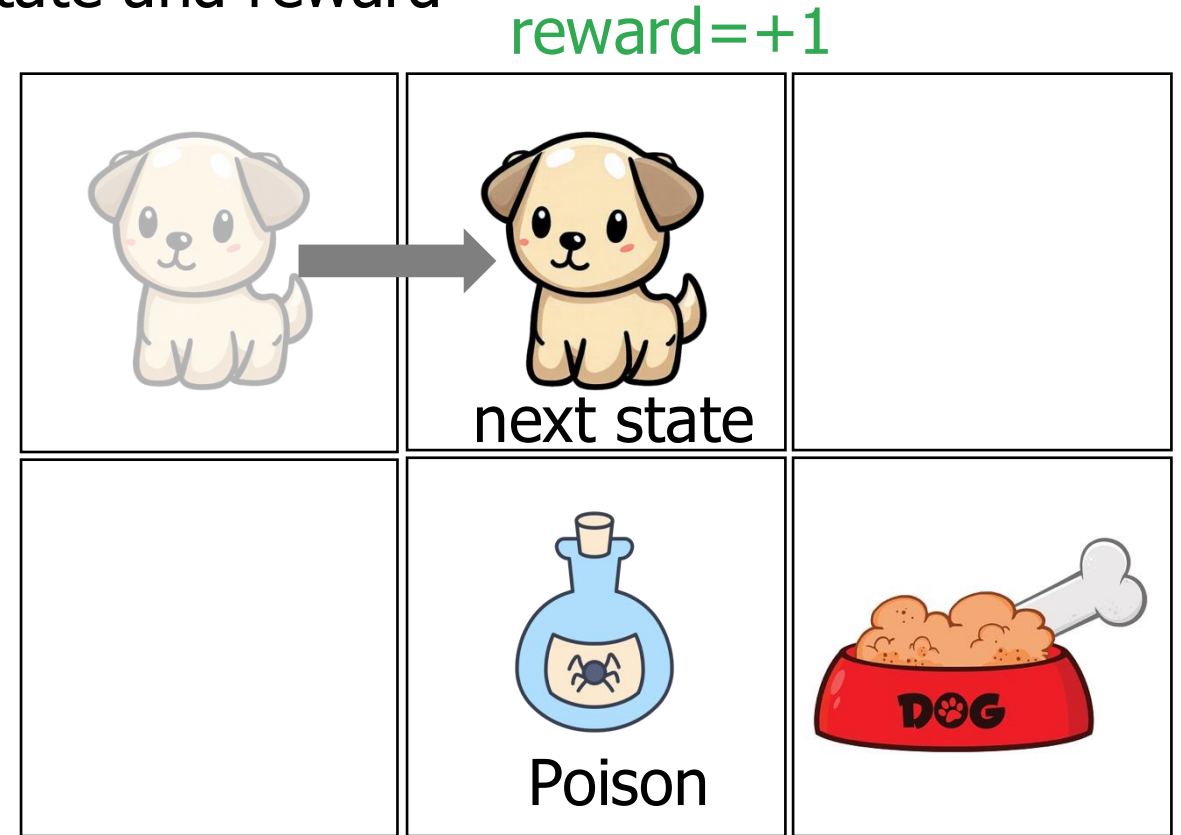
		Actions			
		↑	↓	←	→
States	(0, 0)	0	0	0	0
		0	0	0	0
	(0, 2)	0	0	0	0
	(1, 0)	0	0	0	0
		0	0	0	0
		0	0	0	0

Q-learning: Off-Policy TD Control

- Example:
 - Choose action a based on a lookup of the Q-table
 - Obtain the next state and reward

	↑	↓	←	→
(0, 0)	0	0	0	0
	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-table



Environment

Q-learning: Off-Policy TD Control

- Example:

Update the Q-value

$$Q(s, a) \leftarrow Q(s, a) + \alpha (R(s, a) + \underbrace{\gamma \max_{a'} Q(s', a')}_{\text{Target value}} - Q(s, a))$$

Learning rate

New Q-value estimation Previous Q-value estimation Immediate reward Discounted estimated optimal next Q-value Previous Q-value estimation

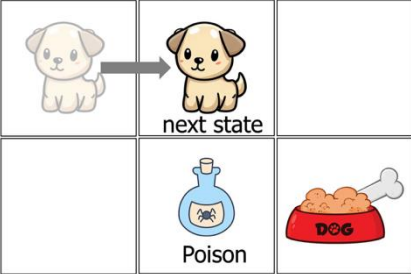
$$Q(s = (0,0), a = \rightarrow) = 0 + 0.1 \times [1 + 0.99 \times 0 - 0] = 0.1 \quad (2)$$

• Example:

- Choose action a based on a lookup of the Q-table
- Obtain the next state and reward

	↑	↓	←	→
(0, 0)	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0

reward=+1



- learning rate $\alpha = 0.1$
- discount factor $\gamma = 0.99$

Q-learning: Off-Policy TD Control

- Example:

Update the Q-value

	↑	↓	←	→
(0, 0)	0	0	0	0
	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-learning: Off-Policy TD Control

- Example:

Update the Q-value

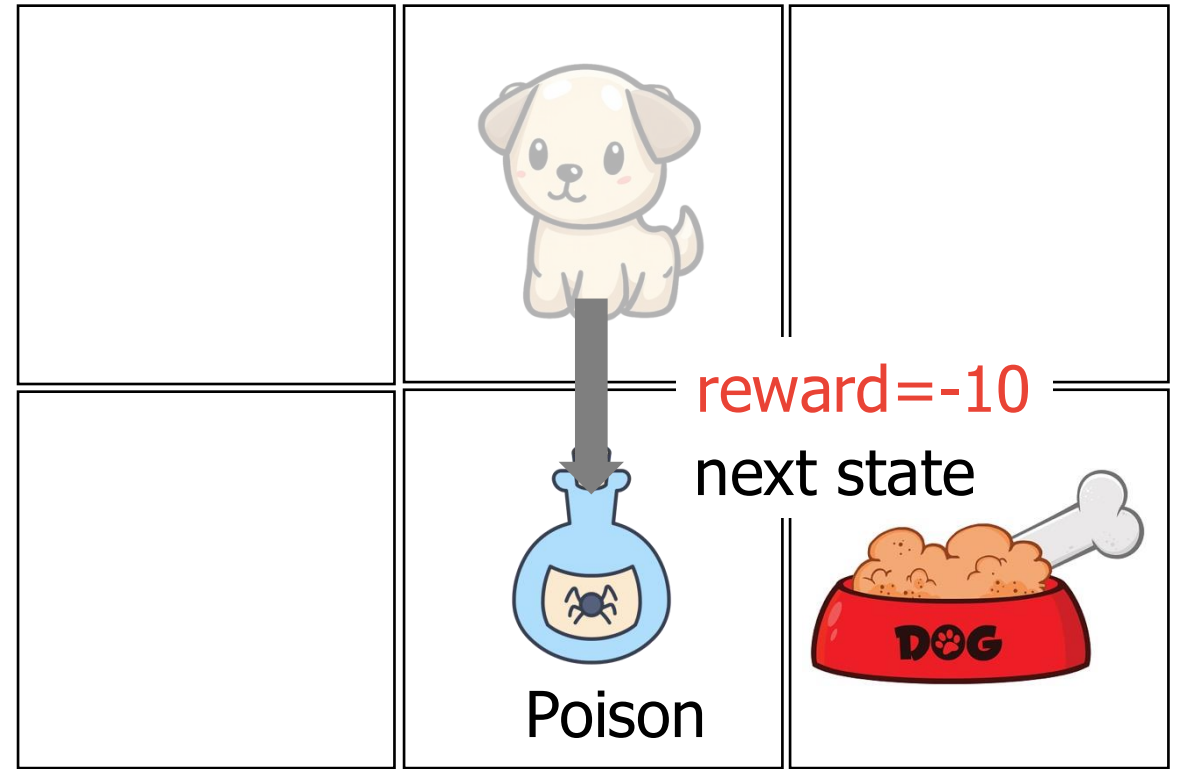
	↑	↓	←	→
(0, 0)	0	0	0	0.1
	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-learning: Off-Policy TD Control

- Example:
 - Choose action a based on a lookup of the Q-table
 - Obtain the next state and reward

	↑	↓	←	→
(0, 0)	0	0	0	0.1
	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-table



Environment

Q-learning: Off-Policy TD Control

- Example:

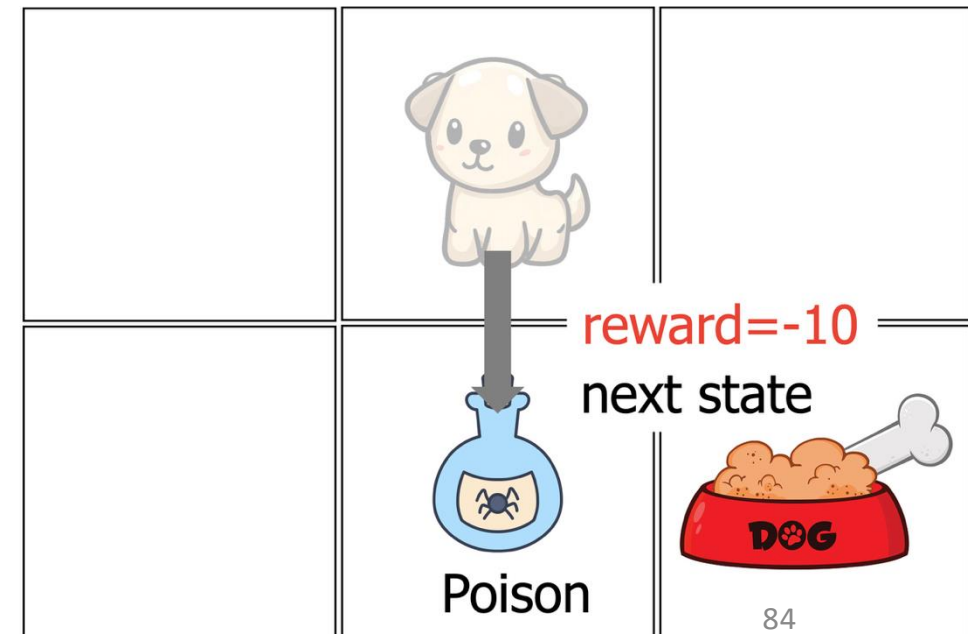
	↑	↓	←	→
🍖	0	-1	0	0

$$Q(s = (0,1), a = \downarrow) = 0 + 0.1 \times [-10 + 0.99 \times 0 - 0] = -1$$

Update the Q-value

✓ The agent gets smarter!

	↑	↓	←	→
(0, 0)	0	0	0	0.1
🍖	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
🧪	0	0	0	0
🍖	0	0	0	0



Q-learning: Off-Policy TD Control

- Example:

Reinitialize a new episode, and continue learning until convergence...

	↑	↓	←	→
(0, 0)	0	0	0	0.1
	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-table



Environment

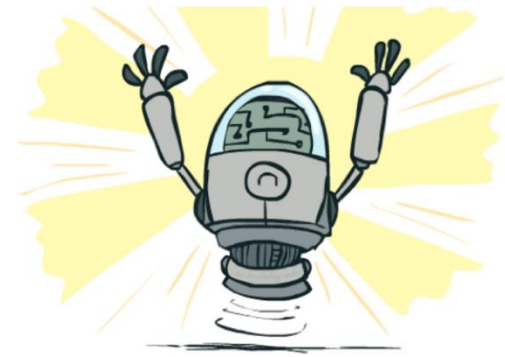
On-Policy vs Off-Policy

Behavior Policy: policy used to navigate in an environment

Target Policy: policy used to optimize decision making

- On-policy: behavior policy should be the same as the target policy
 - Uses the same policy to generate actions and learn from them
 - Analogy: like a chef who learns only by tasting their own cooking
 - Example: SARSA
 - In on-policy learning, the optimal value function is learned from actions taken using the current policy $\pi(a|s)$.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

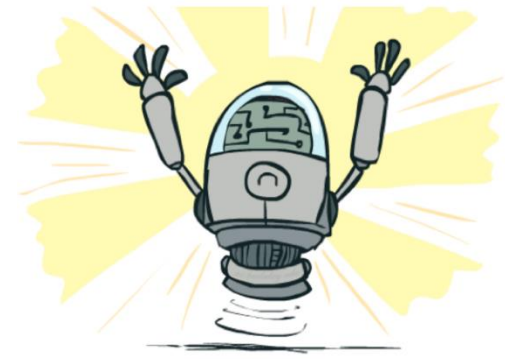


On-Policy vs Off-Policy

Behavior Policy: policy used to navigate in an environment

Target Policy: policy used to optimize decision making

- Off-policy: behavior policy can be different from target policy
 - Learns from one policy (target) while collecting data using another (behavior)
 - Analogy: like a chef who learns by watching others cook and improve recipes based on observations
 - Example: Q-learning



Q-learning: Off-Policy TD Control

- Amazing result: Q-learning converges to optimal policy -- even if you are acting sub-optimally!
- Caveats:
 - You have to explore enough
 - You have to eventually make the learning rate small enough
 - ... but not decrease it too quickly



Technical requirements:

- ☐ All states and actions are visited infinitely often
 - Basically, in the limit, it does not matter how you select actions (!)
- ☐ Learning rate schedule such that for all state and action pairs (s, a) :

$$\sum_{t=0}^{\infty} \alpha_t(s, a) = \infty \quad \sum_{t=0}^{\infty} \alpha_t^2(s, a) < \infty$$

Q-learning: Off-Policy TD Control

Model-free

Does not require the knowledge of the model

Convergence

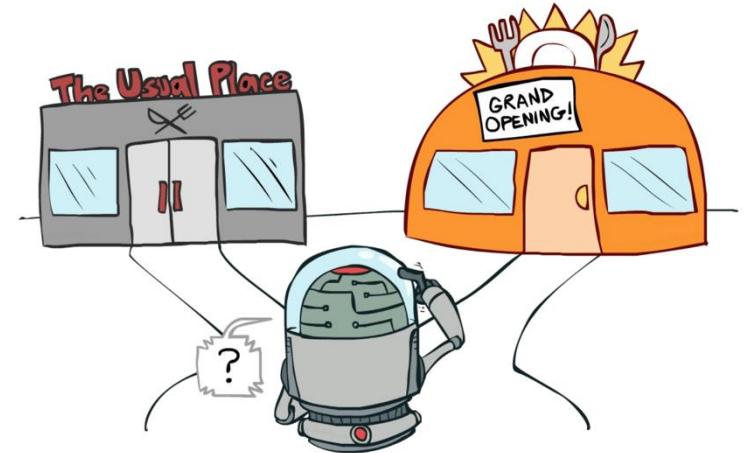
- Every (s, a) is visited infinitely often
- Decaying learning rates condition

- Does not matter how actions are selected

- **Exploration/exploitation strategy**

- ϵ -greedy
 - w.p. ϵ : act randomly (**exploration**)
 - w.p. $1 - \epsilon$: act according to the current greedy policy (**exploitation**)

(Can lower ϵ over time)



Off-policy

Sample-efficient

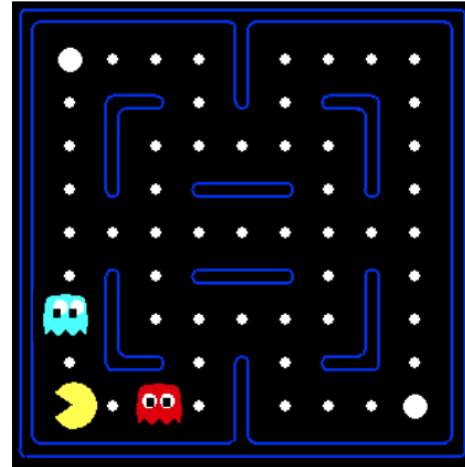
Q-learning

Too many states

- 1 Start with an initial **Q-function** → *Uses explicit $|S| \times |A|$ to represent Q*
- 2 For episode $k = 1, 2, \dots$ till convergence
- 3 Initialize the environment to get the initial state s
- 4 For step $t = 1, 2, \dots$ till termination
- 5 Sample action a from a policy derived from Q
- 6 Take action a , observe next state s' and get reward r
- 7 Perform temporal difference update by
$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a))$$
- 8 Transition to next state $s \leftarrow s'$

Q-learning

Discovered



Undiscovered



Undiscovered

Deep Q-Networks

How to scale Q-learning to practical complex and high-dimensional tasks?

2018 ACM A.M. Turing Award
Deep Learning, Neural Networks



Yoshua Bengio



Geoffrey Hinton



Yann LeCun

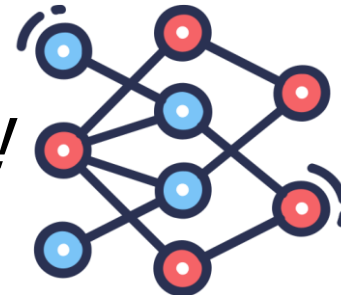
LETTER

doi:10.1038/nature14236

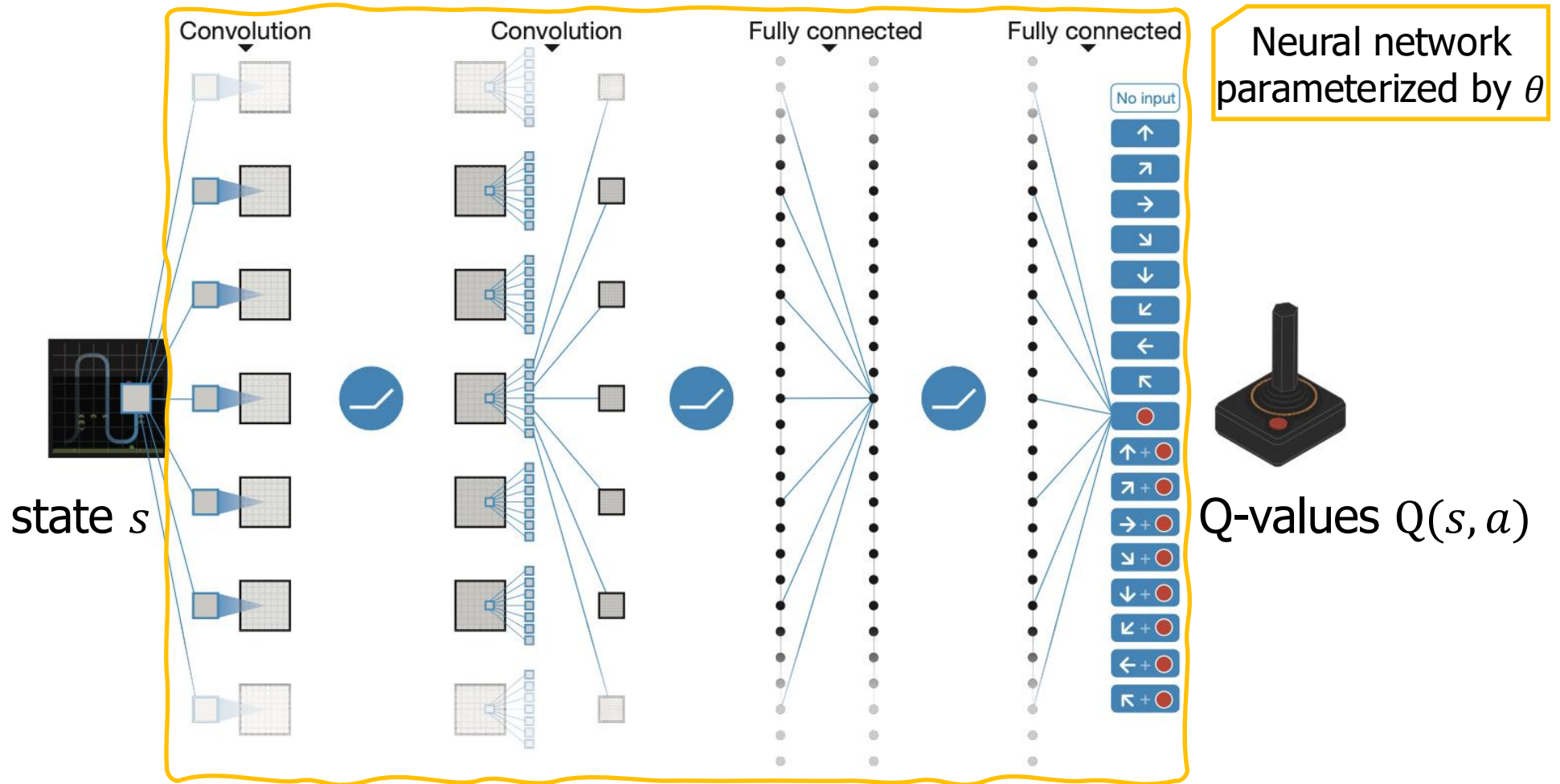
Human-level control through deep reinforcement learning

Volodymyr Mnih^{1*}, Koray Kavukcuoglu^{1*}, David Silver^{1*}, Andrei A. Rusu¹, Joel Veness¹, Marc G. Bellemare¹, Alex Graves¹, Martin Riedmiller¹, Andreas K. Fidjeland¹, Georg Ostrovski¹, Stig Petersen¹, Charles Beattie¹, Amir Sadik¹, Ioannis Antonoglou¹, Helen King¹, Dharshan Kumaran¹, Daan Wierstra¹, Shane Legg¹ & Demis Hassabis¹

+ *Neural networks!*



Deep Q-Networks



$$\text{Loss function: } \mathcal{L}(\theta) = \mathbb{E}_{s,a,r,s' \sim \mathcal{D}} \left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta)}_{\text{target}} - Q(s, a; \theta) \right)^2$$

Part 1: Summary

- Probability Basics
- Bandits and Markov Decision Process
- Value Functions and Bellman Equations
- Dynamic Programming
 - Value Iteration
 - Policy Iteration: Policy Evaluation, Policy Improvement
- Monte-Carlo Methods
 - Monte-Carlo Prediction
 - Monte-Carlo Control
- Temporal Difference Learning
 - Temporal Difference Prediction
 - Temporal Difference Control
 - On-policy SARSA
 - Off-policy Q-learning

For Mid-Term Exam

- Bandits and Markov Decision Process
- Value Functions and Bellman Equations
- Dynamic Programming
 - Value Iteration
 - Policy Iteration: Policy Evaluation, Policy Improvement
- Monte-Carlo Methods
 - Monte-Carlo Prediction
 - Monte-Carlo Control
- Temporal Difference Learning
 - Temporal Difference Prediction
 - Temporal Difference Control
 - On-policy SARSA
 - Off-policy Q-learning

Thank
You

